

技术文档

月探物联——全自主智能搬运系统

学校名称： 武汉理工大学

团队名称： 关关红温队

队长： 袁宇博

队员 1： 曹俊熙

队员 2： 邓英泽

2025 年 12 月

目录

摘 要.....	IV
第一章 项目概述.....	5
1.1 项目背景与目标.....	5
1.1.1 背景介绍.....	5
1.1.2 核心问题.....	5
1.1.3 项目目标.....	5
1.2 技术路线.....	6
1.2.1 任务分析.....	6
1.2.2 功能模块.....	6
1.2.3 程序流程设计.....	7
第二章 功能设计.....	11
2.1 系统总体架构.....	11
2.1.1 任务解析与决策层.....	11
2.1.2 感知与环境建模层.....	11
2.1.3 导航与机械臂执行层.....	12
2.1.4 仿真环境与可视化层.....	12
2.2 系统集成与调试.....	13
2.2.1 自然语言任务解析与 JSON 任务建模.....	14
2.2.2 YOLOv8 视觉识别与物块位姿估计.....	15
2.2.3 MPP1 局部规划与 LSTM 动态障碍预测.....	16
2.2.4 机械臂抓取规划与执行控制.....	18
第三章 功能实现.....	20
3.1 模块实现细节.....	20
3.1.1 自然语言任务解析与 JSON 任务建模模块 (tools_demo).....	20
3.1.2 任务调度与导航协同控制模块.....	20
3.1.3 动态障碍物仿真与轨迹预测模块.....	21
3.1.4 物块视觉识别与位姿获取模块.....	22
3.1.5 机械臂抓取与放置模块.....	23
3.1.6 Foxglove 可视化与评分接口.....	24
3.2 系统集成与调试.....	24
3.2.1 模块间通信方式.....	24
3.2.2 程序运行指令和流程.....	25
3.2.3 集成调试步骤与问题解决策略.....	27

第四章 关键模块源代码29

4.1 自然语言任务解析模块关键代码（TOOLS_DEMO） 29

4.2 动态障碍物解析决策层 35

4.3 解析型预测节点 47

4.4 YOLOv8 视觉识别模块关键代码（YOLOv8_VISION） 59

参考文献61

月探物联——全自主智能搬运系统

摘 要

本作品围绕“月探物联——全自主智能搬运系统”主题，基于大模型技术创新赛提供的 ROS2 仿真平台，构建了一套集大模型指令解析、计算机视觉识别、移动导航规划与机械臂抓取执行于一体的自主作业系统。系统采用轻量化 Llama 大模型完成自然语言任务解析，将模糊指令转换为标准化任务结构；视觉模块创新地使用了 YOLOv8 模型结合 HSV 特征与轮廓检测，实现红蓝目标包裹的实时识别；导航模块基于 Nav2 行为树，MPPI 规划器+LSTM 预测节点来进行路径规划与动态避障，完成机器人在复杂场景中的自主移动；机械臂模块通过 MoveIt2 精确执行抓取与放置动作，并与导航流程实现闭环协同。

整个系统形成“感知—决策—执行—反馈”的自主作业链路，并通过 Foxglove Studio 实时展示指令解析、目标识别、路径规划、抓取状态及任务进度，满足比赛对可视化与实时反馈的要求。在限时 5 分钟内，机器人能够完成多批次指定货物的识别、搬运与放置任务，展现出较高的稳定性、鲁棒性与工程应用价值，为月球仓储基地等未来场景的智能物流作业提供可行的技术方案。

关键词：YOLOv8 计算机视觉；Nav2 自主导航；MPPI 局部规划器；LSTM 动态障碍预测

第一章 项目概述

1.1 项目背景与目标

1.1.1 背景介绍

随着深空探测与月球资源开发技术的发展，月球仓储基地逐渐成为未来航天物流体系的关键节点。其建设需要机器人具备自主识别、自主搬运、精准抓取以及交互协同能力，以确保在高效、低差错率的前提下完成物资分拣与搬运作业。全球校园人工智能算法精英大赛·大模型技术创新赛为此提供标准化平台，要求参赛队伍基于 ROS2、大模型推理、计算机视觉、导航规划及机械臂抓取技术，设计一套智能化、可闭环执行的全自主作业系统。本项目正是围绕此背景展开，旨在构建具备多源感知、语义理解与自主操作能力的智能搬运系统。

1.1.2 核心问题

本项目需解决月球仓储作业过程中的多项关键技术难题，包括：

- (1) 如何在复杂仿真环境中实现红蓝包裹的高精度视觉识别与定位；
- (2) 如何将自然语言任务指令转换为机器人可执行的标准化语义结构；
- (3) 如何使移动机器人在避障环境中自主完成路径规划并精准抵达目标；
- (4) 如何保障机械臂在抓取与放置过程中的稳定性、精度与流程协同；
- (5) 如何构建“感知—决策—执行—反馈”全链路闭环以满足比赛限时需求。

1.1.3 项目目标

- 目标 1：构建稳定高精度的视觉识别模块，实现红/蓝包裹的实时识别与位姿估计。
- 目标 2：利用轻量化大模型完成自然语言指令解析，输出标准化 JSON 任务结构。
- 目标 3：实现移动机器人基于 Nav2 的自主导航、避障及任务点精准到位。
- 目标 4：构建机械臂抓取系统，实现目标物体的精准抓取、搬运与放置。
- 目标 5：完成 Foxglove Studio 可视化布局，实现作业状态、路径规划、抓取情况等多项实时反馈。
- 目标 6：形成可在 5 分钟比赛限时内稳定完成多批次搬运任务的智能作业系统。

1.2 技术路线

1.2.1 任务分析

系统整体按照“感知 → 解析 → 决策 → 执行 → 可视化反馈”五大任务链路进行设计：

- (1) **视觉感知任务**：利用相机和 YOLOv8 完成目标检测、颜色识别与坐标提取，为导航与抓取提供准确的空间信息。
- (2) **指令解析任务**：采用本地部署的 Llama 大模型，将自然语言任务转化为标准化 JSON 命令，实现任务类别、数量与放置区域的结构化表达。
- (3) **路径规划任务**：基于 Nav2 行为树完成 SLAM 建图、全局路径规划与动态避障，确保机器人安全高效地到达抓取与放置区域。
- (4) **机械臂作业任务**：利用 MoveIt2 完成抓取、举升与放置动作，实现多关节协同与轨迹优化，提高作业稳定性。
- (5) **任务协同与可视化**：通过 ROS2 话题机制实现多模块数据交互，并利用 Foxglove Studio 呈现指令、识别、路径与抓取状态，实现任务闭环与可视化评分要求。

1.2.2 功能模块

- (1) **tools_demo**：自然语言任务解析模块，接收中文指令并调用本地大模型，将任务转为标准 JSON 结构。
- (2) **bot_navigation**：任务调度与导航控制模块，按 JSON 任务组织导航流程，并通过话题与机械臂进行协同。
- (3) **auto_move_obstacle&lstm_predictor_node**：动态障碍物仿真与轨迹预测模块，驱动仿真中移动障碍物，并发布其未来运动轨迹供 MPPI 避障使用。
- (4) **yolov8_vision**：物块视觉识别与位姿获取模块，基于 YOLOv8 的图像识别与目标检测模块，输出目标物块位置与类别。
- (5) **auto_grasp_moveit**：机械臂抓取与放置模块，基于 MoveIt 规划机械臂轨迹，结合 LinkAttacher 完成物块抓取和放置。
- (6) **foxglove_dashboard**：数据同步与可视化模块，汇总指令解析、目标识别、导航路径与抓取状态，在 Foxglove 中统一展示系统运行过程。

1.2.3 程序流程设计

本系统采用 分层式架构 + 模块化流程调度 + 数据驱动的 ROS2 通信机制 来实现“感知—理解—决策—执行—反馈”的完整闭环流程。整体流程自上而下划分为任务层、决策层、执行层、感知层、可视化层，通过 ROS2 Topic、Service、Action 进行交互，确保结构清晰、耦合度低、运行可靠，系统整体流程图如图 1-1 所示。

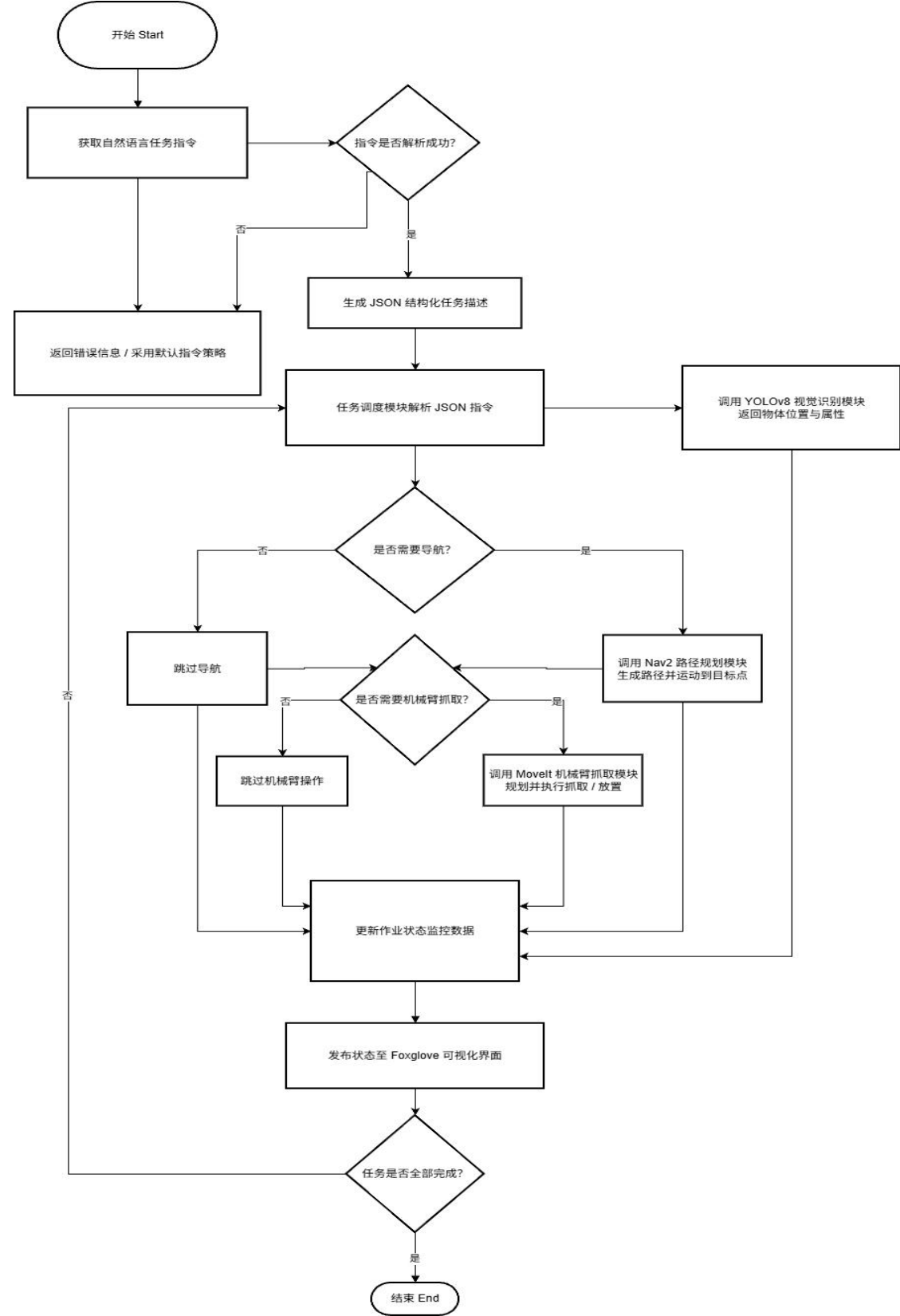


图 1-1 整体流程图

(1) 程序层级结构

本系统在 ROS2 框架下采用分层架构实现，各功能包在不同层级承担清晰的职责，并通过标准化接口进行解耦。最上层为指令与任务定义层，主要由本地部署的 Llama 指令解析模块及其封装节点构成。该层面向用户暴露自然语言接口，接收自由文本形式的作业描述，并通过 llama.cpp 推理将非结构化指令转换为 JSON 任务模型。转换后的任务模型以 ROS2 消息的形式向下游发布，不直接参与任何执行控制。

中间层为任务调度与状态管理层，由 MissionController 节点实现，是整个系统的软件中枢。该节点内部集成有限状态机与任务队列管理逻辑，一方面负责解析来自指令层的 JSON 任务，将其拆解为可执行的子任务单元；另一方面通过调用 Nav2、YOLOv8 视觉节点与 MoveIt2 机械臂规划接口实现跨功能包的行为编排。调度器通过 ROS2 服务、Action 以及话题订阅获取各执行模块的实时状态，将其统一映射为内部状态机的转移条件，从而在“待命、导航、识别、抓取、放置、恢复”等多个状态之间进行切换。

底层为执行与感知层，主要包括三个子系统：基于 YOLOv8 的视觉识别包 yolov8_vision，基于 Nav2 的移动底盘导航与避障系统（内部集成 MPPI 控制器及自研 mppi_lstm_critic 动态障碍物代价函数），以及基于 MoveIt2 的机械臂作业包 auto_grasp_moveit。视觉子系统负责从 /camera/image_raw 等传感器话题中读取图像并输出目标物体的类别与位姿信息；导航子系统在接收到 MissionController 下发的目标位姿后完成路径规划、速度控制与动态障碍物绕行；机械臂子系统在得到抓取/放置位姿后完成碰撞检测、逆解求解与轨迹执行。三者仅通过标准 ROS2 接口与调度层交互，不直接互相依赖，从软件工程角度保证了模块化与可替换性。

在上述纵向分层之外，系统还设置了一条横向的监控与可视化层。MissionController 将任务进度、路径信息、识别结果与抓取状态以统一的话题形式发布，Foxglove 订阅这些话题并构建可视化布局，实现对各层级运行状态的集中展示。通过这种纵向分层、横向监控的组织方式，系统在工程上形成了结构清晰、边界明确、易于调试和扩展的整体软件体系。

(2) 数据流设计

数据流设计以“任务驱动”和“模块解耦”为原则，在 ROS2 通信模型之上构建多条相对独立但互相约束的传输通道，使任务信息、感知数据与执行状态在系统内部构成闭环。

在任务数据通路上,自然语言指令首先由指令解析节点接收并转交给 llama.cpp 模型推理,输出的 JSON 任务结构通过自定义消息封装后发布至 MissionController。该节点不对任务内容做二次修改,而是将其视为只读的任务约束源,在内部生成任务队列和状态机初始条件。任务数据因此在系统中扮演“配置”角色,被贯穿性地引用于导航目标生成、抓取次数控制以及放置区域选择等环节。

在视觉数据通路上,摄像头驱动节点持续向 /camera/image_raw 话题发布图像帧,yolov8_vision 节点订阅该话题并在被调度层触发时执行推理;推理结果以检测数组消息形式输出,其中包含物体类别、二维边界框与投影到机器人坐标系下的三维位姿。MissionController 在接收到有效检测结果后,将位姿信息写入内部黑板,并据此生成导航和机械臂的目标位姿。为降低带宽占用与计算负载,视觉推理并非一直运行,而是由调度层在到达识别区域后以服务或控制话题方式触发一次性推理,从而形成“按需感知”的数据流模式。

在导航与动态障碍物相关的数据通路上,MissionController 通过 Nav2 的 NavigateToPose Action 下发目标位姿,Nav2 在内部利用代价地图、全局规划器与 MPPI 控制器生成速度控制指令,并将执行进度、当前路径与是否到达等信息作为 Action 反馈返回调度器。同时,动态障碍物插件 auto_move_obstacle 在 Gazebo 中周期性发布障碍物位姿,LSTM 预测节点基于这些历史轨迹数据推断未来运动趋势,并通过自定义话题向 mppi_lstm_critic 提供预测结果,后者再将预测到的风险区域编码为代价函数叠加到 MPPI 评分中。这样,导航控制链路形成了“任务目标 → 路径规划 → 预测避障 → 反馈状态”的数据闭环。

在机械臂作业数据通路上,MissionController 将由视觉与任务约束共同确定的抓取/放置位姿通过 MoveIt2 的 MoveGroup Action 下发,MoveIt2 在当前 Planning Scene 中进行路径规划并输出轨迹;轨迹执行过程由 FollowJointTrajectory Action 控制,其结果(是否成功到达、夹爪状态等)被作为 Action 结果回传给 MissionController。调度器根据这些反馈更新任务执行状态,并在必要时触发重规划或恢复策略。

最后,在状态监控与可视化数据通路上,MissionController 将上述各条主干数据流中的关键摘要信息(例如当前任务索引、目标物体属性、底盘轨迹、机械臂抓取结果等)统一封装为监控消息并发布。Foxglove 通过订阅这些话题构建多视图布局,将任务级状态、导航路径、检测标注与关节曲线同步呈现,为调试与裁判评分提供直接的数据支撑。

通过上述数据流设计,系统在工程上实现了任务数据、感知数据与控制数据的分层组织和有序流动,使各功能包既能独立验证,又能在 MissionController 的调度下形成整体一致的行为,从而保证整个作业过程的可控性与可追踪性。

(3) 控制流总体设计

每一子任务完成后, MissionController 根据状态机判断是否需要处理下一个任务单元: 若当前任务执行成功, 则推进任务指针; 若识别失败、抓取失败或路径不可达, 则进入 RECOVERY 状态, 根据异常类型选择重新识别、重新规划、避障重导航或任务中止。所有任务单元执行完毕并满足终止条件后, 系统切换回 IDLE 状态, 等待下一轮自然语言任务输入。

该控制流在工程上形成一个以数据依赖为核心、以行为树为结构、以状态机为调度机制的完整闭环, 使语言解析、视觉感知、移动导航与机械臂控制四类能力在统一逻辑下协同运行, 具有可复现性、可追踪性与可扩展性。

第二章 功能设计

2.1 系统总体架构

本系统采用由上到下的分层架构，自然语言任务解析、任务决策调度、环境感知建模、导航与避障、机械臂抓取以及仿真与可视化各自形成相对独立的功能层，通过 ROS2 话题接口实现解耦。整体数据流从“任务输入→大模型解析→任务队列→导航/抓取执行→仿真反馈”，控制流由中枢节点 MissionController 统一下发和闭环管理，系统架构图如图 2-1 所示。

2.1.1 任务解析与决策层

裁判或用户首先在“任务输入层”给出中文指令（例如“把 2 个红色包裹搬到 A 区，3 个蓝色包裹搬到 C 区”）。指令经由大模型任务解析层中的 llama.cpp 本地大模型服务处理，task_parser 节点将大模型回复解析为结构化 JSON，对任务的颜色 color、数量 count、目标区域 zone 及优先级 priority 做显式建模，形成 JSON 任务队列 {color, count, zone, priority}。任务决策与调度层由 MissionController 节点承担核心逻辑。该节点订阅 JSON 任务队列，将每一条任务实例化为内部任务状态机，根据当前执行进度和机器人状态，分别生成两类控制指令：1) 面向导航的 navigate_to_pose 目标位姿，用于驱动移动底盘接近目标货物或指定区域；2) 面向机械臂的抓取任务描述 {color, zone}，作为抓取规划的输入参数。MissionController 也是系统的“全局黑板”，统一维护当前任务轮次、已完成任务数以及异常恢复策略。

2.1.2 感知与环境建模层

感知与环境建模层负责将仿真环境中的原始传感器数据转换为可供决策与规划使用的语义信息。相机通过话题 /camera/image_raw 发布图像数据，yolov8_vision 节点对该话题进行订阅，利用训练好的 YOLOv8 模型输出目标货物和动态障碍物的检测结果（类别、像素位置以及三维位姿等），并将其封装为 ROS2 消息供上层使用。

与此同时，激光雷达通过 /scan 发布二维激光数据，静态地图由地图服务节点提供，AMCL 节点结合里程计信息完成定位，输出机器人在全局坐标系下的位姿。MissionController 同时订阅 YOLOv8 识别结果和 AMCL 位姿，从而在任务层面获得“机器人在哪、货物在哪、障碍物在哪”的统一环境描述，后续导航规划依赖的局部/全局代价地图由 Nav2 在此基础上自动构建。

2.1.3 导航与机械臂执行层

导航与避障层基于 Nav2 框架实现。MissionController 将导航目标以 `navigate_to_pose` 形式下发给 Nav2，Nav2 内部的全局规划器在静态地图上生成一条从当前 AMCL 位姿到目标区域的全局路径，并结合代价地图进行安全性约束。局部控制采用 MPPI 控制器插件，其中嵌入自定义的 LSTM 动态障碍物 Critic（`mppi_lstm_critic` 包）。该 Critic 利用历史轨迹数据预测动态障碍运动趋势，并在 MPPI 采样过程中对不安全轨迹施加惩罚，实现对移动障碍的前瞻性避让。最终，MPPI 控制器输出连续速度控制指令 `/cmd_vel`，驱动底盘实现平滑的避障导航。机械臂规划与执行层由 MoveIt2 及自定义抓取功能包 `auto_grasp_moveit` 组成。MissionController 在识别到目标货物已处于机械臂工作空间内时，向抓取模块发送 `{color, zone}` 抓取任务。`auto_grasp_moveit` 节点根据当前机械臂状态、目标物体位姿以及抓取约束条件生成抓取轨迹，并调用 `/arm_controller` 接口下发关节轨迹或笛卡尔轨迹。机械臂完成夹取后，系统再次调用导航层将货物运送至 A/B/C 指定区域，最后在目标区域附近触发放置动作，实现“导航—抓取—搬运—放置”的闭环任务链。

2.1.4 仿真环境与可视化层

仿真与可视化层为比赛场景和评审提供统一的观测窗口。Gazebo 仿真环境中构建了月球仓储场景及 A/B/C 放置区域，并通过动态障碍物插件模拟移动障碍。底盘和机械臂执行的 `/cmd_vel` 以及 `/arm_controller` 命令均作用于 Gazebo 中的机器人模型，使仿真状态与实际控制指令保持一致。为了满足大赛对评分系统的要求，系统部署 `rosbridge_server` 将 ROS2 话题桥接为 WebSocket，Foxglove Studio 基于此构建可视化布局。Foxglove 订阅任务解析结果、YOLOv8 识别输出、Nav2 路径规划结果、机械臂抓取状态以及关键状态文本，对“当前任务指令、目标位置、导航路径、抓取与放置过程以及任务完成情况”等信息进行统一展示，既方便调试，又为裁判提供直观的评分依据。通过上述分层设计与清晰的数据/控制流组织，本系统在保证模块解耦的前提下，将大模型任务解析、视觉识别、导航避障和机械臂抓取有机地串联起来，形成一套可在仿真环境中稳定运行的月球仓储智能作业系统。

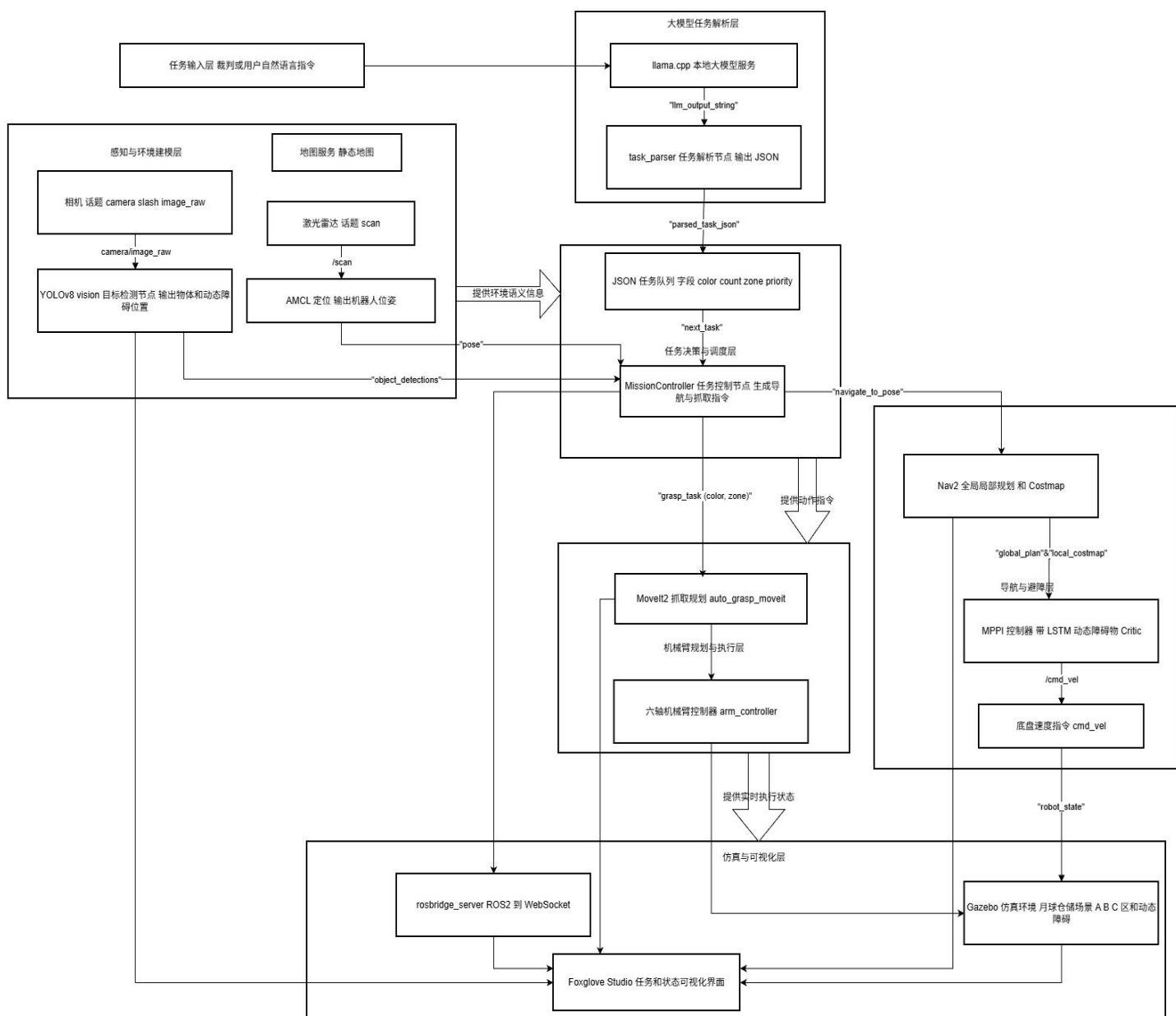


图 2-1 系统架构图

2.2 系统集成与调试

本系统的关键技术围绕“任务语义解析—环境感知与建模—动态避障导航—机械臂抓取执行”四个环节展开，实现从自然语言任务到机器人多子系统协同执行的端到端闭环。各项技术均在 ROS2 通信框架下以独立节点形式实现，通过 Topic / Service / Action 接口完成解耦集成，并由任务调度节点在行为树与状态机的统一调度下运行。

2.2.1 自然语言任务解析与 JSON 任务建模

自然语言任务解析模块基于本地部署的 Llama 大模型和 llama.cpp 推理引擎实现，将裁判或用户给出的中文指令转换为机器人可执行的结构化任务描述。ROS2 中由 task_parser 节点负责与大模型进程交互，其核心流程包括指令接收、提示词构造、推理调用、结果解析与合法性校验。

在工程实现上，task_parser 节点通过订阅“任务输入层”的文本话题或服务请求获得原始指令，将其嵌入预定义 Prompt 模板中，例如要求模型输出包含 {color, count, zone, priority} 字段的 JSON 结构。推理结果以字符串形式返回后，节点在本地执行 JSON 解析与模式校验，对字段缺失、类型不匹配或值越界等情况进行检测，并在必要时采用默认策略（例如缺省数量、默认区域）或向上游反馈解析失败状态。合法的任务被封装为自定义 ROS2 消息并发布至 MissionController，使任务解析完全独立于后端控制逻辑。

任务解析伪代码如图 2-2 所示

```
ps

function parse_command_loop():
    while rclcpp.ok():
        cmd_text = wait_for_command()           // 订阅或同步等待文本指令
        prompt   = build_prompt(cmd_text)       // 构造带约束的提示词模板
        llm_reply = llama_infer(prompt)         // 调用 llama.cpp 进行推理
        json_obj  = safe_json_parse(llm_reply)  // 解析并捕获异常

        if not validate_schema(json_obj):
            publish_parse_error(cmd_text)       // 通知上游解析失败
            continue

        tasks_msg = convert_to_ros_msg(json_obj) // 映射为任务消息
        publish(tasks_msg)                     // 发布至 MissionController
```

图 2-2 任务解析伪代码

通过上述设计，任务解析模块在语义层完成自然语言到 JSON 任务模型的映射，并通过严格的结构校验保证下游任务调度与执行逻辑的可预期性和安全性。

2.2.2 YOLOv8 视觉识别与物块位姿估计

视觉识别模块 `yolov8_vision` 以 RGB 摄像头为输入，通过 YOLOv8 模型实现红蓝包裹的实时检测与定位，并进一步给出机器人坐标系下的目标位姿，为导航与机械臂抓取提供统一空间参考。ROS2 中，摄像头驱动节点持续向 `/camera/image_raw` 话题发布图像帧，`yolov8_vision` 节点订阅该话题并在调度层触发时执行推理，而非持续全帧处理，以减少计算负载。

在算法层面，`yolov8_vision` 对输入图像进行尺寸缩放与归一化预处理，将其送入已经离线训练好的 YOLOv8 模型，输出包含目标类别、置信度和二维边界框的检测结果。结合相机内参、外参以及地面约束或标定板信息，在节点内部完成从图像平面到机器人基座坐标系的坐标变换，得到每个物块的三维位置和朝向。节点还会对置信度过低或尺寸异常的检测结果进行过滤，并在同一任务周期内对历史检测进行时间平滑，以抑制感知噪声。

视觉识别与位姿估计过程可概括为如图 2-3 的视觉识别与位姿估计伪代码。

```
pseudo

function vision_inference_once():
    image = wait_for_image(/camera/image_raw)
    detections_2d = yolov8_forward(image)

    poses_3d = []
    for det in detections_2d:
        if det.conf < THRESHOLD:
            continue
        pixel_box = det.bbox
        pose = project_to_base_link(pixel_box, camera_model, ground_plane)
        poses_3d.append({det.class_id, pose})

    publish_detections(poses_3d) // 自定义检测数组消息
```

图 2-3 视觉识别与位姿估计伪代码

通过上述流程，系统实现了从原始图像到“目标类别+三维位姿”的稳定输出，并以统一消息接口向 `MissionController` 提供任务级可用的环境感知信息。

2.2.3 MPPI 局部规划与 LSTM 动态障碍预测

本项目的动态避障模块以“面向未来的预测式导航”为总体设计理念，通过将动态障碍物建模、轨迹预测、风险评估与行为决策有机耦合，构建一个具有前瞻性的时空推理能力的导航体系。其核心创新在于采用“LSTM 参数回归 + 解析式轨迹生成”的混合建模框架：由 LSTM 捕捉障碍物运动的潜在规律，将复杂的动态行为压缩为低维参数表示，而未来轨迹完全通过解析模型推演得出。这种方式从根本上提升了预测的稳定性与可解释性，使系统能够精确描述往返运动、周期性摆动及 S 型扰动等结构化行为，突破了传统端到端轨迹预测无法外推、易震荡、不可解释的局限。

在此基础上，系统通过定制化的 MPPI LSTM Dynamic Obstacle Critic 将预测结果融入局部规划，构建具备方向性、时序性与行为偏好特征的动态代价函数。该 Critic 不仅识别硬碰撞风险，更能够量化轨迹在障碍物未来路径中的停留时间，并通过速度向量夹角刻画“抢道”与“避让”关系，使机器人具备类似人类驾驶的策略意识，如提前侧绕、等待空隙或延迟入场等。此外，代价模型完全以数学表达定义，可确保系统在参数变化下保持一致行为风格，形成可调可解释的策略空间。综合而言，整个算法结构实现了从“被动避障”向“主动决策”的跃迁，使机器人能够基于未来时空布局进行规划，这正是本项目在动态导航领域的核心技术创新。

在系统集成与工程实践过程中，本项目严格遵循分层设计原则，使预测、评估、行为管理与底盘执行形成相互独立但紧密协作的结构。所有模块通过 ROS2 的话题接口解耦，保证数据流透明、调试路径清晰，同时便于在不同硬件条件下部署与迁移。预测模块在 CPU 上即可稳定运行，通过固定长度滑动窗口和批处理结构确保延迟可控；MPPI 层采用矢量化处理，可在极短时间内对数千条候选轨迹进行风险评分；行为管理器则以轻量几何规则和前向仿真机制作为最终安全保障，使系统即使在高动态环境、密集障碍物或瞬态冲突场景中仍具备优良的容错性与鲁棒性。

工程调试过程中，团队按“从预测准确性 → 规划风险模型 → 行为鲁棒策略”的顺序逐层标定系统参数，使模块间边界清晰、职责明确。最终形成的避障流程具有强逻辑性和可解释性：首先由预测节点持续生成未来轨迹，提供环境的时空结构；随后 MPPI 根据这一结构在代价空间中筛选出最具安全性与通过性的轨迹；当系统遇到预测无法覆盖的极端情境或存在明显时间冲突时，行为层立即接管，采用 WAIT、让行或虚拟探测策略确保系统不失控；最后再由控制链路将安全速度下发至底盘。这一流程不仅体现了多层冗余的工程思想，也保证机器人在实际运行中能够做到“能预见、能决策、能兜底”，从而在复杂动态环境下保持稳定、连续与可信赖的避障能力。

决策层核心代码逻辑如图 2-4-1 所示

```
230
237 # A) 先判定：机器人当前位置是否本身就在动障未来轨道上
238 robot_on_path = self._is_robot_on_future_obstacle_path(
239     x0, y0, self.last_obstacles)
240
241 # B) 选取用于碰撞预测的“仿真速度指令”
242 #
243 # - 正常 CRUISE 下：用 Nav2 输出的 last_cmd
244 # - 当前处于 WAIT 时：用一个沿当前 yaw 的虚拟探测速度 (probe_speed)
245 sim_cmd = self.last_cmd
246 if self.current_mode == "WAIT":
247     sim_cmd = Twist()
248     sim_cmd.linear.x = self.probe_speed
249     sim_cmd.angular.z = 0.0
250     self.get_logger().debug(
251         f"[BehaviorManager] WAIT 模式下使用探测速度进行预测: v={self.probe_speed:.2f} m/s"
252     )
253
254 # C) 沿 sim_cmd 做 horizon 内前向仿真，看会不会撞进 danger_radius (安全硬闸门)
255 will_hit = self._predict_collision(
256     x0, y0, yaw0, sim_cmd, self.last_obstacles
257 )
258
259 # 逻辑：
260 # - 如果将来会撞，且当前不在动障轨道上 → WAIT (刹车就能避免)
261 # - 如果当前就在轨道上 → 不用 WAIT，交给 MPPI critic 去寻找侧向绕行方案
262 if will_hit and not robot_on_path:
263     proposed_mode = "WAIT"
264 else:
265     proposed_mode = "CRUISE"
266
267 # D) gap 等待：前方一小段“走廊”是否会被动障占用
268 if self.enable_gap_wait and proposed_mode != "WAIT" and not robot_on_path:
269     if self._is_forward_corridor_blocked(x0, y0, yaw0, self.last_obstacles):
270         proposed_mode = "WAIT"
271
```

图 2-4-1 决策层核心代码逻辑片段

MPPI 自定义插件核心代码逻辑如图 2-4-2 所示

```
403
404 // 1) 碰撞检测：进入 danger_radius 视为必撞
405 if (dist <= danger_r) {
406     has_collision = true;
407     earliest_collision_time =
408         std::min(earliest_collision_time, t_now);
409     // 不 break：继续统计也无妨，但最终会用 hard penalty
410 }
411
412 // 2) tube 内 / tube 外：记录第一次离开 tube 的时间 t_exit
413 if (in_tube) {
414     if (dist > tube_r) {
415         in_tube = false;
416         t_exit = t_now;
417     }
418 }
419
420 // 3) 方向偏好统计：只在 tube 内且有明显相对运动时才计算 cos(theta)
421 if (dist <= tube_r &&
422     speed > eps_vel &&
423     best_obs_speed > eps_vel) {
424     const double dot = vrx * best_vox + vry * best_voy;
425     const double denom = speed * best_obs_speed;
426     double cos_theta = dot / denom;
427     // 数值安全的 clamp
428     if (cos_theta > 1.0) {
429         cos_theta = 1.0;
430     } else if (cos_theta < -1.0) {
431         cos_theta = -1.0;
432     }
433
434     cos_acc += cos_theta;
435     ++cos_count;
436 }
437
438 // for t

```

图 2-4-2 自定义插件核心代码逻辑片段

MPPI 控制与 LSTM Critic 的交互可抽象为如图 2-4 的伪代码。

```
pseudo

function mppi_control_step(current_state, global_path):
    samples = sample_control_sequences(N, H)           // 采样 N 条长度 H 的控制序列
    predicted_obstacles = query_lstm_predictions()     // 获取未来障碍物分布

    best_u = None
    best_cost = +INF

    for u_seq in samples:
        traj = simulate_dynamics(current_state, u_seq)
        cost = path_cost(traj, global_path)
        cost += dynamic_obstacle_cost(traj, predicted_obstacles) // LSTM Critic 代价
        cost += constraint_cost(traj)

        if cost < best_cost:
            best_cost = cost
            best_u = u_seq[0]                           // 仅执行第一步控制

    publish_cmd_vel(best_u)
```

图 2-4 MPPI 控制结合 LSTM Critic 伪代码

2.2.4 机械臂抓取规划与执行控制

机械臂子系统基于 MoveIt2 实现轨迹规划与运动控制，由 auto_grasp_moveit 功能包对抓取与放置流程进行封装。MissionController 在确认目标物块位于机械臂工作空间并具有可靠位姿后，通过 MoveGroup Action 下发抓取或放置请求，请求中包含由视觉模块输出并结合任务约束计算得到的目标位姿及抓取策略参数。

在规划阶段，MoveIt2 在当前规划场景下执行碰撞检测和逆运动学求解，生成满足关节限制与障碍约束的候选轨迹，并根据路径长度、平滑度和可行性对轨迹进行排序，选取最优解后交给 FollowJointTrajectory Action 执行。抓取过程配合 LinkAttacher 插件完成夹爪与目标刚性连接；放置过程则在到达目标区域附近后，沿预定义下降轨迹缓慢降低高度，确保物块稳定释放并解除虚拟约束。

机械臂动作流程可简要表示为如图 2-5 所示的伪代码。

pseudo

```
function execute_grasp_task(target_pose, zone):  
    approach_pose = offset_pose(target_pose, approach_offset)  
    plan1 = moveit_plan(current_state, approach_pose)  
    execute(plan1)  
  
    grasp_pose = target_pose  
    plan2 = moveit_plan(current_state, grasp_pose)  
    execute(plan2)  
    close_gripper()  
    attach_object()  
  
    place_pose = compute_place_pose(zone)  
    plan3 = moveit_plan(current_state, place_pose)  
    execute(plan3)  
    open_gripper()  
    detach_object()
```

图 2-5 机械臂抓取流程伪代码

通过将视觉位姿、任务约束和规划场景统一纳入 MoveIt2 规划流程，机械臂模块实现了从抓取接近、夹取、搬运到放置的完整动作链条，并通过 Action 反馈与 MissionController 状态机联动，为上层任务调度提供可靠的执行结果与异常信息。

上述关键技术统一的 ROS2 系统架构下实现模块化集成，结合 2.1 节给出的总体架构和数据/控制流设计，使自然语言任务解析、视觉感知、动态避障导航与机械臂抓取在工程上形成结构清晰、边界明确且便于调试与扩展的机器人软件系统。

第三章 功能实现

3.1 模块实现细节

为满足大模型技术创新赛“指令解析—目标识别—导航避障—抓取放置—状态反馈”的整体流程要求，本项目最终形成了如下六个核心实现模块：

3.1.1 自然语言任务解析与 JSON 任务建模模块（tools_demo）

代码位置：yzbot/tools_demo/tools_demo/chat_interactive_ros.py

节点名称：DualModeRobotTaskNode

该节点通过 HTTP 与本地部署的 llama.cpp/Qwen 模型服务交互，实现中文自然语言任务到标准 JSON 结构的转换，并将结果发布到 ROS2 话题 robot_task。

实现要点：

（1）节点启动时读取系统提示词 system_prompt，明确约束大模型输出格式，如：请将任务解析为：{"tasks":[{"color":"red|blue","count":N,"zone":"A|B|C"}]}。

（2）在 parse_task() 中使用 requests.post 向 llama-server 发送请求，设置较小 `temperature` 与超时限制，减少幻觉输出并保证实时性。

```
81 def parse_task(self, user_input):
82     """共享指令解析函数，调用Llama，返回有效任务列表，提示信息"""
83     try:
84         response = requests.post(
85             self.llama_server_url,
86             json={
87                 "model": "deepseek-ai-DeepSeek-RT-Distill-Qwen-32B-jinja",
88                 "messages": [{"role": "system", "content": self.system_prompt}, {"role": "user", "content": user_input}],
89                 "max_tokens": 1024, "temperature": 0.0, "timeout": 10
90             },
91             headers={"Content-Type": "application/json"})
92     except requests.exceptions.ConnectionError:
93         return [], "Llama模型未连接，请先启动llama-server"
94     except json.JSONDecodeError:
95         return [], "模型输出格式错误（非JSON）"
96     except Exception as e:
97         return [], f"解析失败: {str(e)}"
```

（3）对模型返回的字符串尝试 json.loads，若解析失败或字段缺失，则补全默认值，并在 task_feedback 话题上给出错误原因，方便在 Foxglove 中排查。

3.1.2 任务调度与导航协同控制模块

代码位置：yzbot/bot_navigation/bot_navigation/mission_controller.py

节点名称：MissionController

该节点是整个系统的“中枢大脑”，负责：

订阅 robot_task，解析任务队列；

1)订阅/object_detection/object_poses 获取物块与仓位位姿；

2)调用 Nav2 (BasicNavigator) 完成移动底盘导航；

3)通过同步话题与机械臂抓取模块协同工作；

4)发布多路状态话题供 Foxglove 可视化。

主要话题接口：

订阅接口：

1)robot_task (String)：自然语言解析后的 JSON 任务；

2)/object_detection/object_poses` (mybot/msg/ObjectPoseArray)：红/蓝方块、A/B/C 仓位、动态障碍物与机器人自身的位姿信息；

3)/amcl_pose (PoseWithCovarianceStamped)：机器人当前位姿；

4)/task_sync_arm2nav (String)：机械臂完成抓取或放置的反馈。

发布接口：

1)/pick_target_id (String)：当前需要抓取的方块 ID；

2)/task_sync_nav2arm (String)：通知机械臂“可以抓取/可以放置”；

3)/mission_state、/nav_status、/arm_status、/nav_target_name (String)：任务阶段、导航状态、机械臂状态与当前目标名称等。

核心控制逻辑采用“任务队列 + 就近优先”的循环机制：首先对解析后的任务列表（每个任务包含目标方块颜色、抓取数量及放置仓位信息）依次迭代，在单个任务的执行过程中，会针对需抓取的每个方块，先从对应颜色的方块缓存中筛选出距离机器人当前位置最近且未被使用的目标方块 (target)，随后调用 nav_to_block_front (target) 函数规划并导航至方块前方的安全位姿；导航成功后，通过 pub_pick_id 发布目标方块的 ID，同时通过 pub_nav2arm 发布“nav_done_grasp”信号通知机械臂启动抓取动作，待接收来自 /task_sync_arm2nav 话题的“grasp_done”反馈后，再调用 nav_to_zone (task.zone) 导航至指定放置仓位前方，接着通过 pub_nav2arm 发布“nav_done_place”信号触发机械臂放置动作，直至收到“place_done”反馈即完成单次方块搬运。在此过程中，系统会实时更新已使用方块的 ID 集合 (used_ids)，并通过 /mission_state 话题反馈任务执行进度。为避免导航线程阻塞与回调函数之间的相互干扰，该节点采用 MultiThreadedExecutor 执行器与 ReentrantCallbackGroup 回调组组合的设计，有效提升了并发处理性能，即便在多话题高频更新的场景下，仍能保持稳定运行。

3.1.3 动态障碍物仿真与轨迹预测模块)

(1) Gazebo 动态障碍物插件

代码位置：auto_move_obstacle/auto_move_obstacle.cc

插件名称：AutoMoveObstacle` (ModelPlugin)

在 world SDF 中为特定模型挂载该插件，并通过<start_x> / <end_x> / <speed>等参数配置直线往返和 S 形轨迹。插件在 Load()中读取参数并在 OnUpdate()回调中根据仿真时间更新模型位置，实现可重复、可调速的动态障碍物，用于算法调试与预测验证。

(2) 解析式轨迹预测节点

代码位置：yzbot/bot_navigation/scripts/lstm_predictor_node.py

节点名称：ObstacleTrajectoryPredictor

该节点订阅 /object_detection/object_poses，只关注 type == "obstacle_dynamic"的目标，通过两帧之间的位移估算当前速度，并结合“往返线段”运动模型，在未来时间窗口内解析式计算障碍物轨迹，发布到/object_trajectory`

(mybot/msg/ObstacleTrajectoryArray)。

核心思想：用 LSTM 订阅物体位姿节点，并通过神经网络解析出物体运动轨迹。

(3) MPPI LSTM 动态障碍物 Critic

代码位置：mppi_lstm_critic/src/lstm_dynamic_obstacle_critic.cpp

插件配置：mppi_lstm_critic/lstm_dynamic_obstacle_critic_plugin.xml

在 Nav2 的 MPPI 控制器参数文件（如 originbot_nav2b.yaml）中，将

LSTMDynamicObstacleCritic 作为额外 critic 加入 critics 列表。该 critic 在每次滚动优化时读取 `/object\_trajectory` 中的预测轨迹，将靠近动态障碍物未来路径的样本轨迹赋予更高代价，实现“提前绕行”的规划效果，从而满足比赛中“机器人成功完成对可移动物体的避障”的评分要求。

3.1.4 物块视觉识别与位姿获取模块

(1) Gazebo 世界位姿采集插件

代码位置：yzbot/mybot/gazebo_object_pose_plugin.cpp

插件类型：WorldPlugin

插件在 Gazebo 世界加载时启动一个 ROS2 节点 gazebo_object_pose_node，遍历仿真世界中的模型列表，将名称中包含 red_cube/blue_cube 的模型标记为方块，将 zone_a/b/c 标记为仓位，将 six_arm 标记为机器人本体，统一封装为 mybot/msg/ObjectPoseArray 并以 10 Hz 频率发布到/object_detection/object_poses。该消息中包含：

- 1) id: 模型名称（如 red_cube_1、zone_A）；
- 2) type: block_red/block_blue/ zone / robot;
- 3) pose: 在 map 坐标系下的三维位姿。

MissionController 通过该话题即可获得所有方块和仓位的精确位姿，无需在视觉节点中手动做三维重建，大幅降低实现难度。

(2) YOLOv8 目标检测节点

代码位置：yolov8_vision/yolov8_vision/yolo_node.py

节点名称：YoloV8Node

节点基于 Ultralytics YOLOv8 实现红/蓝立方体的端到端检测，接收 Image 消息，完成推理并发布带检测框叠加的图像到/yolov8/annotated_image。在当前仿真平台中，物体三维位置仍由 Gazebo 插件直接给出，而 YOLOv8 主要用于验证模型效果与为后续实机迁移预留接口。

3.1.5 机械臂抓取与放置模块

位于 yzbot/mybot/auto_grasp_moveit.py 的 AutoGraspMoveItNode 节点，以 MoveIt2 的 MoveGroup 动作接口为核心运动控制支撑，结合 IFRA_LinkAttacher 吸附服务的物体绑定能力，实现了六轴机械臂对目标方块的精准抓取与指定位置放置功能，为机器人方块搬运任务提供了可靠的末端执行层支持。

订阅接口：

1)/task_sync_nav2arm (String)：由 MissionController 发送的“导航已完成，可抓取/可放置”信号，取值如 nav_done_grasp、nav_done_place；

2)/pick_target_id (String)：当前需要抓取的方块 ID，用于选择挂接目标。

发布接口：

1)/task_sync_arm2nav (String)：向导航模块反馈 grasp_done、place_done 等状态；

2)/arm_state (String)：对外发布机械臂当前状态（空闲、抓取中、放置中等）。服务：

3)/link_attacher_node/attach、/link_attacher_node/detach：与 Gazebo 物理引擎交互，实现夹爪与方块的吸附/释放。

节点内部采用严格的状态机模型管理机械臂全流程动作，核心状态流转以空闲状态为起点：当通过 /task_sync_nav2arm 话题收到导航节点发送的“nav_done_grasp”抓取就绪指令时，状态机先切换至抓取中，随后调用预设的抓取位姿参数，通过 MoveIt! 规划器生成关节空间轨迹，按 0.6 的速度 / 加速度缩放因子执行运动；待机械臂到达抓取位姿后，调用 LinkAttacher 的 ATTACHLINK 服务完成目标方块与末端执行器（link6）的吸附绑定，最后返回初始位姿（arm_pose_init），状态机正式切换至抓取完成（ARM_STATE_GRASP_DONE），并通过 /task_sync_arm2nav 反馈“arm_done”信号。

若在抓取完成状态下收到“nav_done_place”放置就绪指令，状态机先切换至放置中，重复抓取阶段的位姿生成、轨迹规划与执行流程，到达放置位姿后调用 DETACHLINK 服务解除方块绑定，再返回初始位姿，最终状态机切回空闲状态并反馈执行完成信号；且放置指令存在严格的状态校验，仅当当前为抓取完成状态时才会执行，避免无效指令触发异常。

为保障系统稳定性，节点设计了多重防护机制：通过线程锁（threading.Lock）保护当前状态变量（current_arm_state），防止工作线程与回调线程并发修改冲突；MoveIt! 规划与执行分别设置 5 秒、20 秒超时，LinkAttacher 服务调用设置 5 秒超时，目标方块 ID 存在 10 秒超时 fallback 逻辑，避免单一环节阻塞导致系统卡死；同时

采用单元素指令队列（deque）控制指令执行顺序，确保同一时间仅处理一个动作指令，进一步规避动作冲突。

3.1.6 Foxglove 可视化与评分接口

根据大模型技术创新赛评分要求，需要在 Foxglove Studio 中展示指令解析结果、目标识别、路径规划、抓取状态与任务进度等信息。

本项目与 Foxglove 可视化工具的集成围绕核心流程展开，通过订阅系统关键话题实现全链路状态可视、可追溯，具体集成逻辑如下：指令解析与任务状态可视化方面，通过订阅任务发布话题 /robot_task 和双端反馈话题 /task_feedback，借助 Foxglove 的 JSON 查看器和文本组件，直观呈现当前标准化任务列表（如红 / 蓝方块数量、目标区域等）与指令解析状态（包括解析成功、待确认、发布完成或解析失败等），让用户实时掌握任务发起全流程；目标识别与物块定位呈现上，一方面订阅 YOLOv8 检测节点输出的 /yolov8/annotated_image 话题，通过 2D 图像组件查看带边界框的标注结果，另一方面结合 /object_detection/object_poses 话题提供的物块、目标区坐标数据，在 3D 视图中精准还原物块在世界坐标系中的位置，实现视觉检测结果与空间位置的联动展示；导航轨迹可视化则直接复用 Nav2 内置的 /plan（规划路径）和 /trajectory（执行轨迹）话题，在 Foxglove 的 2D 地图组件中叠加显示机器人的预规划路径与实际行驶轨迹，方便直观校验导航精度；抓取与放置状态监控通过整合机械臂状态话题 /arm_state（空闲 / 抓取中 / 放置中等枚举状态）、状态文本话题 /arm_status 及导航 - 机械臂协同话题 /task_sync_nav2arm，利用文本组件和状态指示器，清晰标记每次抓取、放置动作的执行结果（成功或失败）；作业进度反馈层面，通过订阅 /mission_state（整体任务状态）和 /nav_status（导航阶段状态），不仅能展示每一轮方块搬运任务的完成时间，还能通过进度条组件呈现整体任务推进情况，让作业进度一目了然。

3.2 系统集成与调试

3.2.1 模块间通信方式

系统整体采用 ROS2 话题、动作（Action）、服务（Service）与 HTTP 结合的混合通信架构，按“数据传输 - 操作执行 - 跨模块交互”分层设计，既保障各环节的实时性与可靠性，又实现模块间的高效解耦，具体通信逻辑如下：ROS2 话题作为系统数据交互的核心载体，按功能场景分为三类：在上层任务流中，robot_task 与 task_feedback 话题承担 LLM 解析模块、任务控制节点（MissionController）及 Foxglove 可视化工具之间的文本指令与 JSON 格式数据交换，是任务发起、解析反馈的核心链路；在感知与环境交互层面，相机原始图像通过 /camera/image_raw 话

题流向视觉检测节点（yzbot_vision/yolov8_vision），为目标检测提供输入；Gazebo 位姿插件发布的 /object_detection/object_poses 话题，同步输出物块、目标区的空间坐标，供任务控制节点规划路径及轨迹预测节点做运动分析；轨迹预测节点生成的 /obstacle_trajectory 话题则传递给 MPPI Critic，为避障导航提供支撑；在执行与协同环节，导航与机械臂的动作同步通过 /task_sync_nav2arm、/task_sync_arm2nav 话题实现，/pick_target_id 话题负责传递待抓取方块的 ID 信息，而 /mission_state、/nav_status、/arm_status、/nav_target_name、/arm_state 等话题作为对外状态广播通道，实时输出整体任务进度、导航状态、机械臂动作状态及当前导航目标，为 Foxglove 可视化提供数据支撑。

ROS2 动作与服务主要承担需同步响应或长时执行的核心操作：机械臂的轨迹规划与精准执行依赖 MoveIt2 的 MoveGroup 动作接口，确保关节运动的平滑可控；方块的吸附与释放通过 LinkAttacher 服务完成，实现机械臂末端与目标物的快速绑定与解绑；Nav2 底层则通过 FollowPath 等动作接口接收 BasicNavigator 下发的导航目标点，驱动机器人完成路径跟踪与位置校准。

HTTP 接口用于实现 LLM 解析模块与本地大模型的跨进程解耦通信：chat_interactive_ros.py 模块通过 HTTP POST 请求调用 llama-server 服务（接口地址如 http://localhost:8081/v1/chat/completions），将用户自然语言指令传递给大模型进行结构解析，这种设计让大模型可独立部署、维护与升级，无需依赖 ROS2 生态直接运行。

这套分层通信架构让上层任务解析、感知检测、导航规划与机械臂执行等模块实现清晰解耦，既满足比赛场景下的实时性要求，又为后续任一模块的独立替换、功能扩展提供了灵活的适配空间，完全适配系统迭代与场景扩展需求。

3.2.2 程序运行指令和流程

（1）仿真环境与动态障碍物启动

通过 launch 文件加载 Gazebo 仿真场景及自定义插件，命令如下：

```
ros2 launch mybot gazebo_world.launch.py
```

该命令将启动集成动态障碍物生成插件（AutoMoveObstacle）与物体位姿发布插件（ObjectPosePlugin）的仿真世界：AutoMoveObstacle 负责自动生成随机运动的动态障碍物，模拟复杂作业环境；ObjectPosePlugin 实时采集场景内目标物（红 / 蓝方块、A/B/C 目标区、机器人本体）的空间位姿，并通过/object_detection/object_poses 话题发布，为后续感知与导航提供环境数据支撑。

（2）导航与定位栈启动

启动基于 Nav2 的自主导航与 AMCL 定位系统，命令如下：

```
ros2 launch bot_navigation nav_bringup_gazebo.launch.py \
```

```
params_file:=param/originbot_nav2b.yaml use_sim_time:=true
```

其中，params_file 指定导航系统配置文件（包含路径规划算法参数、AMCL 定位 covariance 配置、速度限制等），use_sim_time:=true 启用仿真时间同步，确保导航

模块与 Gazebo 仿真环境的时间戳一致性。该步骤启动后，系统将完成 Nav2 核心节点（规划器、控制器、恢复行为节点）与 AMCL 定位节点的初始化，具备自主路径规划与定位能力。

（3）轨迹预测节点启动

启动基于 LSTM 的动态障碍物轨迹预测节点，命令如下：`ros2 run bot_navigation lstm_predictor_node.py`

该节点订阅/object_detection/object_poses 话题获取动态障碍物实时位姿，通过预训练 LSTM 模型预测其未来运动轨迹，并将预测结果通过/obstacle_trajectory 话题发布至 MPPI Critic 模块，为导航系统提供避障决策依据，提升动态环境下的导航鲁棒性。

（4）视觉检测模块启动

通过 launch 文件启动 YOLOv8 目标检测模块，命令如下：

`ros2 launch yolov8_vision yolov8_vision.launch.py`

该模块启动后，将订阅相机原始图像话题/camera/image_raw，通过预训练 YOLOv8 模型实现红 / 蓝方块、目标区的实时检测，输出带边界框的标注图像（话题/yolov8/annotated_image），同时为后续任务调度提供目标物视觉确认支撑。

（5）机械臂抓取模块启动

启动基于 MoveIt2 的机械臂运动控制与吸附执行模块，命令如下：

`ros2 run mybot auto_grasp_moveit.py`

该节点初始化 MoveIt2 运动规划客户端与 LinkAttacher 服务客户端，通过订阅/task_sync_nav2arm 话题接收导航系统的协同指令，完成机械臂抓取 / 放置位姿规划、目标物吸附（/ATTACHLINK 服务）与释放（/DETACHLINK 服务），并通过/arm_state 话题发布机械臂状态（空闲 / 抓取中 / 放置中）。

（6）任务调度与导航控制启动

启动系统核心任务调度节点，命令如下：`ros2 run bot_navigation mission_controller.py`

该节点作为系统“中枢”，负责接收标准化任务列表、解析任务参数（方块颜色 / 数量、目标区）、协同导航与机械臂动作：通过订阅/robot_task 话题获取任务指令，结合/amcl_pose（机器人定位）与/object_detection/object_poses（目标位姿）规划作业路径，通过/task_sync_nav2arm 与/task_sync_arm2nav 话题实现导航 - 机械臂协同，同时发布/mission_state 等状态话题供可视化监控。

（7）大模型服务与 ROS 网关启动

需开启两个独立终端，分别启动大模型服务与 ROS2-LLM 交互网关：

终端 A：启动 llama.cpp 大模型服务，提供自然语言解析能力：

`llama-server -m ~/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf --port 8081 --threads 8`

其中，-m 指定预训练模型路径，--port 8081 设置服务端口，--threads 8 配置 CPU 推理线程数，启动后提供 HTTP 接口供 ROS 节点调用。

终端 B：启动 ROS2-LLM 交互网关，实现自然语言指令解析：

`ros2 run tools_demo chat_interactive_ros.py`

该网关通过 HTTP POST 请求调用 llama-server，将用户自然语言指令（如“把两个红色包裹搬到 A 区”）解析为标准化 JSON 任务列表，通过/robot_task 话题发布至任务调度节点，同时通过/task_feedback 话题反馈解析状态。

（8）可视化监控启动

启动 Foxglove Studio 工具，通过“ROS2 DDS”数据源连接系统，加载预定义的自定义布局文件(JSON 格式)，该布局已关联系统所有核心状态话题(/mission_state、/nav_status、/arm_state 等)，通过 2D 地图、3D 场景、文本组件、进度条等可视化控件，实时展示机器人位姿、导航轨迹、任务进度与各模块工作状态。系统就绪与任务执行

完成上述 8 个步骤后，系统进入全功能就绪状态。用户可通过 Foxglove Studio 的文本输入组件输入自然语言任务指令（例如“把两个红色包裹搬到 A 区，三个蓝色包裹搬到 C 区”），指令经 LLM 解析为标准化任务后，系统将自动启动作业流程，机器人在 5 分钟内完成目标方块的抓取、导航运输与放置全流程，所有操作状态可通过 Foxglove Studio 实时监控。

3.2.3 集成调试步骤与问题解决策略

为保障系统在比赛 5 分钟限时内稳定完成红蓝方块搬运任务，集成调试采用“单模块验证→子系统联调→全链路压测”的分阶段递进策略，逐步排查模块功能缺陷与协同冲突，确保各环节符合竞赛技术要求。

为夯实系统运行基础，首先开展单模块验证工作：针对大模型交互模块，通过简单脚本直接调用 llama-server，模拟网络中断、指令格式错误等异常场景，验证 chat_interactive_ros.py 网关模块能否返回合理默认结果，避免因解析失败导致任务阻塞；在 Gazebo 仿真环境中固定机器人姿态，逐一核对 gazebo_object_pose_plugin 发布的红蓝方块、A/B/C 目标区坐标与仿真模型实际位置的重合度，确保环境感知数据的准确性；独立运行 auto_grasp_moveit.py 节点，通过手动发布 /test_sync_nav2arm 话题与 /pick_target_id 话题，验证机械臂抓取、放置轨迹的可达性与运动稳定性，排除单模块功能隐患。

单模块功能验证通过后，进入子系统联调阶段，重点校验模块间的协同一致性与数据交互有效性：借助 ros2 topic echo 命令与 rqt_graph 工具，实时监测 robot_task、/object_detection/object_poses、/task_sync_nav2arm、/task_sync_arm2nav 等核心话题的连通状态，确保消息传输无丢失、频率满足实时性要求；针对导航子系统，先在 Nav2 仅加载基础 critics 的配置下验证静态环境导航稳定性，再逐步集成 LSTMDynamicObstacleCritic，观察机器人是否能提前识别动态障碍物预测轨迹带并完成绕避，确保动态环境下的导航鲁棒性；利用 Foxglove Studio 的 time scrubber 功能回放任务执行过程，逐帧核对“自然语言指令解析→路径规划→机械臂抓取→导航运输→放置→状态反馈”的时序逻辑，确保各环节衔接符合设计预期，无时序错乱问题。

在分阶段调试过程中，针对出现的典型问题制定了针对性解决策略：针对 Nav2 启动后无响应的问题，排查发现核心原因是 AMCL 未发布首帧 `/amcl_pose` 导致导航初始化失败，为此在 `MissionController` 节点的 `try_init_nav2()` 方法中增加状态机判断，确保仅在定位数据稳定后才构造 `BasicNavigator` 实例，避免初始化时机不当引发的功能异常；针对动态障碍物代价计算异常的问题，在 `lstm_predictor_node` 中增加轨迹预测失败的保护逻辑，当预测结果缺失时返回空数组，同时在 `Critic` 端优化代价计算逻辑，对无障碍场景快速返回 0 代价，避免无效计算占用系统资源；针对机械臂与导航节奏不一致的问题，通过 `/task_sync_nav2arm` 与 `/task_sync_arm2nav` 话题设置显式同步点，明确导航完成后需等待机械臂动作反馈方可进入下一环节，同时为机械臂抓取、放置动作设置 20 秒超时阈值，避免因机械臂执行异常导致导航模块提前进入下一轮任务，确保两者协同节奏一致。

通过上述分阶段调试与问题修正，系统各模块功能与协同逻辑均达到竞赛要求，能够在规定时间窗内稳定完成指定数量的红蓝方块搬运任务，且所有作业状态均可在 Foxglove Studio 中完整呈现，满足比赛可视化监控需求。

第四章 关键模块源代码

4.1 自然语言任务解析模块关键代码（tools_demo）

自然语言任务解析模块位于 yzbot/tools_demo 功能包中，节点文件为 tools_demo/chat_interactive_ros.py。其核心功能是调用本地大模型服务，将中文指令解析为标准化 JSON 任务列表并通过 ROS2 话题发布。

文件：yzbot/tools_demo/tools_demo/chat_interactive_ros.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import requests
import json
import threading
import time
```

```
# 反馈模板（status: success/error/pending）
```

```
FEEDBACK_TEMPLATE = {
    "status": "",
    "result": "",
    "message": ""
}
```

```
class DualModeRobotTaskNode(Node):
```

```
    def __init__(self):
```

```
        super().__init__("dual_mode_robot_task")
```

```
        # 1. ROS2 话题初始化（Foxglove 交互用）
```

```
        self.task_pub = self.create_publisher(String, "robot_task", 10) # 任务发
```

布

```
        self.foxglove_input_sub = self.create_subscription(
            String, "foxglove_task_input", self.foxglove_input_cb, 10
```

```
        ) # Foxglove 指令输入话题
```

```
        self.foxglove_confirm_sub = self.create_subscription(
```

```
            String, "foxglove_task_confirm", self.foxglove_confirm_cb, 10
```

```
        ) # Foxglove 确认指令话题
```

```
        self.feedback_pub = self.create_publisher(String, "task_feedback", 10) #
```

双端共享反馈话题

2. 共享状态管理（加线程锁，防止两端冲突）

```
self.lock = threading.Lock() # 线程锁
self.pending_task = None # 待确认任务（共享）
self.llama_server_url = "http://localhost:8081/v1/chat/completions"
self.system_prompt = ""
```

仅输出纯 JSON 数组，无任何额外文本。每个任务包含 3 个字段：

- num: 正整数（物块数量）
- color: 仅为 red（红色）/blue（蓝色）
- to: 仅为 A/B/C（目标区域）

输入是用逗号分隔的任意数量任务，输出是所有有效任务的 JSON 数组。

示例：

输入："把 2 个红色放到 A，3 个蓝色放到 B"

输出：[{"color":"red","num":2,"to":"A"}, {"color":"blue","num":3,"to":"B"}]

输入："红 1 到 C，蓝 5 到 A，红 3 到 B，蓝 2 到 C"

输出：[{"color":"red","num":1,"to":"C"}, {"color":"blue","num":5,"to":"A"}, {"color":"red","num":3,"to":"B"}, {"color":"blue","num":2,"to":"C"}]

输入："1 个红色到 C，3 个蓝色到 A，4 个红色到 B，7 个蓝色到 C"

输出：[{"color":"red","num":1,"to":"C"}, {"color":"blue","num":3,"to":"A"}, {"color":"red","num":4,"to":"B"}, {"color":"blue","num":7,"to":"C"}]

3. 启动终端交互子线程（独立于主线程，不阻塞 Foxglove）

```
self.terminal_thread = threading.Thread(target=self.terminal_interactive,
daemon=True)
self.terminal_thread.start()
```

4. 启动提示（双端）

```
self.send_feedback(
    "success",
    message=" 双端交互模块启动！\n 终端：直接输入自然语言指令（输入 quit 退出）\nFoxglove：用「指令输入面板」输入，「确认面板」操作"
)
```

```

def send_feedback(self, status, result="", message=""):
    """双端同步反馈：终端打印 + Foxglove 话题发布"""
    # 1. 生成反馈 JSON（Foxglove 用）
    feedback = FEEDBACK_TEMPLATE.copy()
    feedback["status"] = status
    feedback["result"] = result
    feedback["message"] = message
    feedback_str = json.dumps(feedback, ensure_ascii=False)
    self.feedback_pub.publish(String(data=feedback_str))

    # 2. 终端打印（自然语言格式）
    terminal_msg = f"\n[反馈] {message}"
    if result:
        terminal_msg += f"\n[解析结果] {result}"
    if status == "error":
        self.get_logger().error(terminal_msg)
    else:
        self.get_logger().info(terminal_msg)

def parse_task(self, user_input):
    """共享指令解析函数：调用 Llama，返回（有效任务列表，提示信息）
    """
    try:
        response = requests.post(
            self.llama_server_url,
            json={
                "model":
                    "deepseek-ai-DeepSeek-RT-Distill-Qwen-32B-jinja",
                "messages":
                    [{"role": "system", "content": self.system_prompt}, {"role": "user", "content": user_input}],
                "max_tokens": 1024, "temperature": 0.0, "timeout": 10
            },
            headers={"Content-Type": "application/json"}
        )
        response.raise_for_status()
        task_list = json.loads(response.json())["choices"][0]["message"]["content"].strip()
        valid_tasks = [t for t in task_list if t.get("color") in ["red", "blue"] and
            t.get("to") in ["A", "B", "C"]]
    
```

```

        return valid_tasks, "指令解析成功"
    except requests.exceptions.ConnectionError:
        return [], "Llama 模型未连接（请先启动 llama-server）"
    except json.JSONDecodeError:
        return [], "模型输出格式错误（非 JSON）"
    except Exception as e:
        return [], f"解析失败：{str(e)}"

# ----- 终端交互逻辑 -----
def terminal_interactive(self):
    """终端自然语言交互：输入→解析→确认→发布"""
    while rclpy.ok():
        # 1. 终端输入自然语言指令
        user_input = input("\n 请输入终端指令（输入 quit 退出）：").strip()
        if user_input.lower() == "quit":
            self.send_feedback("success", message=" 终端交互退出")
            break
        if not user_input:
            self.send_feedback("error", message="终端指令为空，请重新输入")
            continue

        # 2. 解析指令（加锁，防止与 Foxglove 冲突）
        with self.lock:
            self.send_feedback("success", message=f"终端正在解析指令：{user_input}")

            valid_tasks, parse_msg = self.parse_task(user_input)

            if not valid_tasks:
                self.send_feedback("error", result="[]", message=f"终端 {parse_msg}（无有效任务）")
                continue

            # 3. 终端确认（自然语言选择 Y/N）
            task_str = json.dumps(valid_tasks, ensure_ascii=False)
            self.send_feedback("pending", result=task_str, message=f"终端解析完成！待确认发布")
            while True:
                confirm = input("是否发布到 robot_task 话题？ [Y/N]：").strip().upper()

```

```

        if confirm in ["Y", "N"]:
            break
        print(" 请输入 Y（确认）或 N（取消）")

# 4. 执行发布/取消
if confirm == "Y":
    self.task_pub.publish(String(data=task_str))
    self.send_feedback("success", result=task_str, message="终端已确认发布任务")

    self.pending_task = None
else:
    self.send_feedback("success", result=task_str, message="终端已取消发布任务")

    self.pending_task = None

# ----- Foxglove 交互逻辑 -----
def foxglove_input_cb(self, msg):
    """处理 Foxglove 输入的自然语言指令"""
    user_input = msg.data.strip()
    if not user_input:
        self.send_feedback("error", message="Foxglove 指令为空，请重新输入")

    return

# 加锁处理，防止与终端冲突
with self.lock:
    # 1. 解析指令
    self.send_feedback("success", message=f"Foxglove 正在解析指令：{user_input}")

    valid_tasks, parse_msg = self.parse_task(user_input)

    if not valid_tasks:
        self.send_feedback("error", result="", message=f"Foxglove{parse_msg}（无有效任务）")
        return

    # 2. 进入待确认状态（共享 pending_task）
    self.pending_task = valid_tasks
    task_str = json.dumps(valid_tasks, ensure_ascii=False)
    self.send_feedback(

```

```

        "pending",
        result=task_str,
        message=f"Foxglove 解析完成！请在「确认面板」点击“确认
发布”或“取消发布”"
    )

def foxglove_confirm_cb(self, msg):
    """处理 Foxglove 的确认指令（中文按钮对应“confirm”/“cancel”）
    """

    confirm_cmd = msg.data.strip().lower()

    # 加锁处理，防止与终端冲突
    with self.lock:
        if self.pending_task is None:
            self.send_feedback("error", message="Foxglove 无待确认任务，
请先输入指令")
            return

        # 执行发布/取消
        task_str = json.dumps(self.pending_task, ensure_ascii=False)
        if confirm_cmd == "confirm":
            self.task_pub.publish(String(data=task_str))
            self.send_feedback("success", result=task_str,
message="Foxglove 已确认发布任务")
            self.pending_task = None
        elif confirm_cmd == "cancel":
            self.send_feedback("success", result=task_str,
message="Foxglove 已取消发布任务")
            self.pending_task = None
        else:
            self.send_feedback("error", message=f"Foxglove 无效指令：
{confirm_cmd}，仅支持 confirm/cancel")

def main(args=None):
    rclpy.init(args=args)
    node = DualModeRobotTaskNode()
    try:
        rclpy.spin(node) # 主线程：监听 Foxglove 话题
    except KeyboardInterrupt:

```

```

        node.send_feedback("success", message=" 程序整体退出")
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

4.2 动态障碍物解析决策层

决策层（behavior_manager_node.py）

此文件根据当前小车位姿与获取的未来位姿进行风险评估，作为小车大脑负责小车的让行和放行

```

# 文件：yzbot/bot_navigation/ bot_navigation / behavior_manager_node.py
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

```

behavior_manager_node.py

行为层（Supervisor）——在 Nav2 / MPPI 之外，再加一层“硬闸门 + 人性化等待”。在本版本中，我们不改 nav2_bringup 的链路，Nav2 仍然是：

```

controller_server ——► /cmd_vel_nav ——► nav2_velocity_smoother ——►
/cmd_vel ——► 底盘

```

BehaviorManager 的工作方式是：

订阅：/cmd_vel_nav （Nav2 控制器输出的速度，用于预测和决策）

订阅：/obstacle_trajectory （动障预测）

发布：/cmd_vel （只有在 WAIT / 危险状态下才发布 0 速度，作为硬刹车）

也就是说：

1. 平时（CRUISE）：不往 /cmd_vel 发布任何速度，让 Nav2 原生控制链路全权接管。
2. 如果沿着「当前速度指令」往前滚动几秒，会进入动态障碍的危险半径：往 /cmd_vel 连续发布 0 速度，进入 WAIT 状态（安全硬闸门）。
3. 如果未来几秒内，机器人前方一小段“走廊”会被动态障碍占用（典型路口 / 会车）：偏向 WAIT，让机器人像人一样“在路口等一等”。
4. 如果机器人当前这个位置本身就在动障未来轨道上：

不再用 WAIT 直接刹死，而是交给 MPPI 的 critic 去选择侧向躲避 / 插空。

```

"""

```

```

import math
from typing import Optional

```

```

import rclpy
from rclpy.node import Node
from rclpy.duration import Duration

from geometry_msgs.msg import Twist
from std_msgs.msg import String
from mybot.msg import ObstacleTrajectoryArray

from tf2_ros import Buffer, TransformListener, TransformException

def yaw_from_quaternion(q):
    """从四元数计算 yaw。"""
    siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
    cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z)
    return math.atan2(siny_cosp, cosy_cosp)

class BehaviorManagerNode(Node):
    def __init__(self):
        super().__init__("behavior_manager")
        # ===== 参数 =====
        # 话题：与当前 nav2 配置对应
        # controller_server: cmd_vel_topic: "cmd_vel_nav"
        # nav2_velocity_smoother: 输入 /cmd_vel_nav, 输出 /cmd_vel (由
navigation_launch.py remap)
        #
        # BehaviorManager:
        #   订阅 /cmd_vel_nav          (看 Nav2 想怎么动)
        #   发布 /cmd_vel              (只在 WAIT 时硬刹车 0 速度)
        self.declare_parameter("cmd_in_topic", "/cmd_vel_nav")
        self.declare_parameter("cmd_out_topic", "/cmd_vel")
        self.declare_parameter("state_topic", "/behavior_state")

        # 安全区
        self.declare_parameter("safety_radius", 0.7)    # 外圈：进入后才开始关注 (目前
没用到, 可后续扩展)
        self.declare_parameter("danger_radius", 0.55)   # 内圈：预测进入就视为危险
        self.declare_parameter("horizon_time", 3.0)    # 预判时间窗口 (秒)
        self.declare_parameter("sim_dt", 0.05)         # 前向仿真步长 (秒)

```

```

# cmd 超时
self.declare_parameter("cmd_timeout", 0.5)      # 超过多久没新 cmd，就当作无
效

# WAIT 持续时间（防止一会儿 WAIT 一会儿 CRUISE 抖动）
self.declare_parameter("min_wait_time", 1.0)    # 至少停 1 秒再考虑放行

# 帧名
self.declare_parameter("global_frame", "map")
self.declare_parameter("base_frame", "base_link")

# ===== 前方走廊 “gap 等待” 参数 =====
self.declare_parameter("enable_gap_wait", True)
self.declare_parameter("gap_front_min", 0.55)
self.declare_parameter("gap_front_max", 2.0)
self.declare_parameter("gap_lateral_half_width", 0.75)

# ===== 当前点是否在动障未来轨道上的判定半径 =====
self.declare_parameter("path_radius", 0.45)

# ===== WAIT 状态下用于 “探测” 的虚拟前向速度 =====
self.declare_parameter("probe_speed", 0.3)

# 读取参数
self.cmd_in_topic =
self.get_parameter("cmd_in_topic").get_parameter_value().string_value
self.cmd_out_topic =
self.get_parameter("cmd_out_topic").get_parameter_value().string_value
self.state_topic =
self.get_parameter("state_topic").get_parameter_value().string_value

self.safety_radius = float(self.get_parameter("safety_radius").value)
self.danger_radius = float(self.get_parameter("danger_radius").value)
self.horizon_time = float(self.get_parameter("horizon_time").value)
self.sim_dt = float(self.get_parameter("sim_dt").value)
self.cmd_timeout = float(self.get_parameter("cmd_timeout").value)
self.min_wait_time = float(self.get_parameter("min_wait_time").value)

self.global_frame =
self.get_parameter("global_frame").get_parameter_value().string_value

```

```

        self.base_frame =
self.get_parameter("base_frame").get_parameter_value().string_value

        self.enable_gap_wait = bool(self.get_parameter("enable_gap_wait").value)
        self.gap_front_min = float(self.get_parameter("gap_front_min").value)
        self.gap_front_max = float(self.get_parameter("gap_front_max").value)
        self.gap_lateral_half_width =
float(self.get_parameter("gap_lateral_half_width").value)
        self.path_radius = float(self.get_parameter("path_radius").value)
        self.probe_speed = float(self.get_parameter("probe_speed").value)
        if self.sim_dt <= 0.0:
            self.sim_dt = 0.05
        # 只打一遍 TF 警告用的 flag (代替 warn_once)
        self.tf_warned = False
        self.get_logger().info(
            "[BehaviorManager] 启动\n"
            f"  cmd_in_topic   = {self.cmd_in_topic}\n"
            f"  cmd_out_topic  = {self.cmd_out_topic}\n"
            f"  state_topic    = {self.state_topic}\n"
            f"  safety_radius  = {self.safety_radius:.3f}\n"
            f"  danger_radius  = {self.danger_radius:.3f}\n"
            f"  horizon_time   = {self.horizon_time:.2f}s, sim_dt={self.sim_dt:.3f}s\n"
            f"  cmd_timeout    = {self.cmd_timeout:.2f}s,
min_wait_time={self.min_wait_time:.2f}s\n"
            f"  frames: {self.global_frame} -> {self.base_frame}\n"
            f"  enable_gap_wait={self.enable_gap_wait}, "
            f"gap_front=[{self.gap_front_min:.2f}, {self.gap_front_max:.2f}], "
            f"gap_lateral_half_width={self.gap_lateral_half_width:.2f}, "
            f"path_radius={self.path_radius:.2f}, "
            f"probe_speed={self.probe_speed:.2f}"
        )
        # ===== TF =====
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        # ===== 状态缓存 =====
        self.last_cmd: Twist = Twist()
        self.last_cmd_time = self.get_clock().now()
        self.has_cmd = False
        self.last_obstacles: Optional[ObstacleTrajectoryArray] = None

```

```

# 行为状态 + 抖动抑制
self.current_mode: str = "CRUISE"    # "CRUISE" or "WAIT"
self.mode_change_time = self.get_clock().now()
# ===== 通讯接口 =====
# Nav2 控制器输出速度 (/cmd_vel_nav)
self.sub_cmd = self.create_subscription(
    Twist,
    self.cmd_in_topic,
    self.cmd_cb,
    20,
)
# 动态障碍预测 (你已有的 /obstacle_trajectory)
self.sub_traj = self.create_subscription(
    ObstacleTrajectoryArray,
    "/obstacle_trajectory",
    self.traj_cb,
    10,
)
# 输出给底盘 (/cmd_vel), 只在 WAIT 时写 0 速度
self.pub_cmd = self.create_publisher(
    Twist,
    self.cmd_out_topic,
    20,
)
# 行为状态 (字符串: CRUISE / WAIT)
self.pub_state = self.create_publisher(
    String,
    self.state_topic,
    10,
)
# 定时器: 固定频率做风险评估 + 下发速度
self.timer = self.create_timer(self.sim_dt, self.timer_cb)
self.get_logger().info("[BehaviorManager] 行为管理节点已就绪。")
# ===== 回调: 收到 cmd_vel_nav ===== #
def cmd_cb(self, msg: Twist):
    self.last_cmd = msg
    self.last_cmd_time = self.get_clock().now()
    self.has_cmd = True

```

```

# ===== 回调：收到动态障碍预测 ===== #
def traj_cb(self, msg: ObstacleTrajectoryArray):
    self.last_obstacles = msg
# ===== 定时器：核心决策逻辑 ===== #
def timer_cb(self):
    now = self.get_clock().now()
    old_mode = self.current_mode # 记录之前的模式
    # ----- 1. 先算“候选模式” ----- #
    proposed_mode = "CRUISE"
    # 1) 没有有效 cmd: 那就停
    if (not self.has_cmd) or ((now - self.last_cmd_time) >
Duration(seconds=self.cmd_timeout)):
        proposed_mode = "WAIT"
    # 2) 没有障碍预测：直接放行（不干预）
    elif self.last_obstacles is None or not self.last_obstacles.trajectories:
        proposed_mode = "CRUISE"
    else:
        # 3) 获取机器人当前位姿
        try:
            tf = self.tf_buffer.lookup_transform(
                self.global_frame,
                self.base_frame,
                rclpy.time.Time()
            )
        except TransformException as ex:
            # 刚启动或 Nav2 刚激活时 TF 可能还没出来，不要让节点崩，放行 cmd
            if not self.tf_warned:
                self.get_logger().warn(
                    f"[BehaviorManager] TF {self.global_frame}->{self.base_frame} 查询失
败: {ex}"
                )
                self.tf_warned = True
            proposed_mode = "CRUISE"
        else:
            x0 = tf.transform.translation.x
            y0 = tf.transform.translation.y
            yaw0 = yaw_from_quaternion(tf.transform.rotation)

```

```

# A) 先判定：机器人当前位置是否本身就在动障未来轨道上
robot_on_path = self._is_robot_on_future_obstacle_path(
    x0, y0, self.last_obstacles)

# B) 选取用于碰撞预测的“仿真速度指令”
#
# - 正常 CRUISE 下：用 Nav2 输出的 last_cmd
# - 当前处于 WAIT 时：用一个沿当前 yaw 的虚拟探测速度
(probe_speed)
sim_cmd = self.last_cmd
if self.current_mode == "WAIT":
    sim_cmd = Twist()
    sim_cmd.linear.x = self.probe_speed
    sim_cmd.angular.z = 0.0
    self.get_logger().debug(
        f'[BehaviorManager] WAIT 模式下使用探测速度进行预测:
v={self.probe_speed:.2f} m/s"
    )

# C) 沿 sim_cmd 做 horizon 内前向仿真，看会不会撞进 danger_radius (安
全硬闸门)
will_hit = self._predict_collision(
    x0, y0, yaw0, sim_cmd, self.last_obstacles
)

# 逻辑：
# - 如果将来会撞，且当前不在动障轨道上 → WAIT (刹车就能避免)
# - 如果当前就在轨道上 → 不用 WAIT，交给 MPPI critic 去寻找侧向
绕行方案
if will_hit and not robot_on_path:
    proposed_mode = "WAIT"
else:
    proposed_mode = "CRUISE"

# D) gap 等待：前方一小段“走廊”是否会被动障占用
if self.enable_gap_wait and proposed_mode != "WAIT" and not robot_on_path:
    if self._is_forward_corridor_blocked(x0, y0, yaw0, self.last_obstacles):
        proposed_mode = "WAIT"

```

```

# ----- 2. 抖动抑制 + 最终模式 ----- #
final_mode = proposed_mode

if old_mode == "WAIT" and proposed_mode == "CRUISE":
    # 从 WAIT 想切回 CRUISE，要看停了多久
    dt = (now - self.mode_change_time).nanoseconds * 1e-9
    if dt < self.min_wait_time:
        final_mode = "WAIT"

if final_mode != old_mode:
    self.current_mode = final_mode
    self.mode_change_time = now
    self.get_logger().info(f"[BehaviorManager] 模式切换: {old_mode} -> {final_mode}")

# ----- 3. 根据 final_mode 下发速度 ----- #
#
# 注意：这里与“串在链路中间”的版本不同：
# - WAIT：直接往 /cmd_vel 写 0，强行刹车（覆盖 nav2_velocity_smoother /
behavior_server）
# - CRUISE：什么都不发，让 Nav2 自己通过 /cmd_vel 控制底盘
if final_mode == "WAIT":
    zero = Twist()
    self.pub_cmd.publish(zero)
else:
    # CRUISE 状态下不发布任何速度，避免与 Nav2 原生输出抢话题
    pass

# ----- 4. 发布状态字符串 ----- #
state_msg = String()
state_msg.data = final_mode
self.pub_state.publish(state_msg)

# ===== 碰撞预测：沿当前 cmd 仿真机器人轨迹 ===== #
def _predict_collision(
    self,
    x0: float,
    y0: float,
    yaw0: float,
    cmd: Twist,

```

```

    obs_array: ObstacleTrajectoryArray,
) -> bool:
    v = cmd.linear.x
    w = cmd.angular.z

    # 如果本来就几乎没动，相当于在等，不交给行为层硬闸门，
    # 让 MPPI + Critic 自己决定是否侧向躲避。
    if abs(v) < 1e-3 and abs(w) < 1e-3:
        return False

    steps = max(1, int(self.horizon_time / self.sim_dt))
    x = x0
    y = y0
    yaw = yaw0

    for step in range(1, steps + 1):
        t_future = step * self.sim_dt

        x += v * math.cos(yaw) * self.sim_dt
        y += v * math.sin(yaw) * self.sim_dt
        yaw += w * self.sim_dt

        min_dist = float("inf")

        for obs_traj in obs_array.trajectories:
            poses = obs_traj.future_poses
            times = obs_traj.future_times
            n = len(poses)
            if n == 0:
                continue

            idx = n - 1
            if len(times) == n:
                for j in range(n):
                    if times[j] >= t_future:
                        idx = j
                        break

            px = poses[idx].position.x
            py = poses[idx].position.y

```

```

        dx = x - px
        dy = y - py
        dist = math.hypot(dx, dy)

        if dist < min_dist:
            min_dist = dist

    if min_dist < self.danger_radius:
        self.get_logger().debug(
            f'[BehaviorManager] 预测 t={t_future:.2f}s 会进入危险圈, '
            f'min_dist={min_dist:.3f} < danger_radius={self.danger_radius:.3f}'
        )
        return True

    return False

# ===== 判断“机器人当前这个点是否在动障未来轨道上” ===== #
def _is_robot_on_future_obstacle_path(
    self,
    x0: float,
    y0: float,
    obs_array: ObstacleTrajectoryArray,
) -> bool:
    if obs_array is None or not obs_array.trajectories:
        return False

    for obs_traj in obs_array.trajectories:
        poses = obs_traj.future_poses
        times = obs_traj.future_times
        n = len(poses)
        if n == 0 or len(times) != n:
            continue

        for pose, t in zip(poses, times):
            if t > self.horizon_time:
                break

        ox = pose.position.x
        oy = pose.position.y

```

```

        dx = ox - x0
        dy = oy - y0
        dist = math.hypot(dx, dy)

        if dist < self.path_radius:
            self.get_logger().debug(
                "[BehaviorManager] robot_on_path: 当前点将在 t=%0.2fs 内被动障扫过,
dist=%0.3f"
                % (t, dist)
            )
            return True

        return False

# ===== 前方走廊是否被动障占用 (gap 等待) ===== #
def _is_forward_corridor_blocked(
    self,
    x0: float,
    y0: float,
    yaw0: float,
    obs_array: ObstacleTrajectoryArray,
) -> bool:
    if obs_array is None or not obs_array.trajectories:
        return False

    cos_yaw = math.cos(yaw0)
    sin_yaw = math.sin(yaw0)

    for obs_traj in obs_array.trajectories:
        poses = obs_traj.future_poses
        times = obs_traj.future_times
        n = len(poses)
        if n == 0 or len(times) != n:
            continue

        for pose, t in zip(poses, times):
            if t > self.horizon_time:
                break

```

```

    ox = pose.position.x
    oy = pose.position.y

    dx = ox - x0
    dy = oy - y0

    # 转到机器人当前 yaw 坐标系:
    forward = dx * cos_yaw + dy * sin_yaw
    lateral = -dx * sin_yaw + dy * cos_yaw

    if forward < self.gap_front_min or forward > self.gap_front_max:
        continue

    if abs(lateral) > self.gap_lateral_half_width:
        continue

    self.get_logger().debug(
        "[BehaviorManager] gap_wait: 前方走廊被动障占用 - "
        f"obs at t={t:.2f}s, forward={forward:.2f}, lateral={lateral:.2f}"
    )
    return True

return False

def main(args=None):
    rclpy.init(args=args)
    node = BehaviorManagerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info("[BehaviorManager] 手动退出")
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

4.3 解析型预测节点

```
# 文件: yzbot/bot_navigation/ scripts / lstm_predictor_node.py
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
"""
```

LSTM 解析参数式动态障碍物轨迹预测节点（自包含版本）

功能:

- 订阅 /object_detection/object_poses (mybot/msg/ObjectPoseArray)
- 只关注 type == "obstacle_dynamic" 的模型（例如 obstacle2 / obstacle3 / obstacle4）
- 为每个障碍物维护最近一段观测轨迹（默认 16 帧）
- 使用训练好的 LSTMThetaModel，从观测 (x,y) 序列回归解析参数 θ :
 $\theta = [\text{axis_flag}, \text{min_val}, \text{max_val}, \text{fixed_coord}, \text{speed}, \text{s_amp}, \text{s_waves}]$
- 再用 θ + 当前最新观测位置，按解析式（往返 + S 型）生成未来若干秒的预测轨迹
- 将结果发布到 /obstacle_trajectory (mybot/msg/ObstacleTrajectoryArray)
- 同时通过日志 + /obstacle_theta_debug 话题输出 θ 供调试

```
"""
```

```
import math
```

```
from typing import Dict, List, Tuple
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
from std_msgs.msg import Header, Float32MultiArray
```

```
from geometry_msgs.msg import Pose
```

```
from mybot.msg import (
    ObjectPoseArray,
    ObstacleTrajectory,
    ObstacleTrajectoryArray,
)
```

```
import torch
```

```
import torch.nn as nn
```

```

# =====
# 自带 LSTMThetaModel 定义（与训练时保持一致）
# =====

class LSTMThetaModel(nn.Module):
    """
    输入: (batch, obs_len, 2)  -> 观测到的历史轨迹 (x, y)
    输出: (batch, 7)           -> 解析参数  $\theta$ :
        [axis_flag, min_val, max_val, fixed, speed, s_amp, s_waves]
    """

    def __init__(
        self,
        input_dim: int = 2,
        hidden_dim: int = 64,
        num_layers: int = 2,
        param_dim: int = 7,
    ):
        super().__init__()
        self.input_dim = input_dim
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers
        self.param_dim = param_dim

        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
        )
        self.fc = nn.Linear(hidden_dim, param_dim)

    def forward(self, obs_seq: torch.Tensor) -> torch.Tensor:
        """
        obs_seq: (batch, obs_len, 2)
        return: (batch, 7)
        """
        lstm_out, (h, c) = self.lstm(obs_seq)          # lstm_out: (B, obs_len, hidden_dim)
        last = lstm_out[:, -1, :]                      # (B, hidden_dim)
        theta = self.fc(last)                          # (B, 7)

```

```

        return theta

# =====
# 主节点
# =====

class LSTMPredictorNode(Node):
    def __init__(self):
        super().__init__("lstm_obstacle_predictor")

        # ----- ROS 参数 -----
        # 预测时间窗口 & 步长
        self.declare_parameter("prediction_horizon", 3.0)    # 预测 3 秒
        self.declare_parameter("prediction_dt", 0.1)        # 每 0.1s 一个预测点
        self.declare_parameter("frame_id", "map")           # 假定 map 与
world 对齐

        # LSTM 权重路径（放在 bot_navigation 包内部）
        self.declare_parameter(
            "model_path",
            "/home/ros/dev_ws/src/yzbot/training/models/theta_lstm.pth",
        )

        # 是否打印  $\theta$  调试信息
        self.declare_parameter("debug_theta", True)

        self.prediction_horizon = (

self.get_parameter("prediction_horizon").get_parameter_value().double_value
        )
        self.prediction_dt = (
            self.get_parameter("prediction_dt").get_parameter_value().double_value
        )
        self.frame_id = (
            self.get_parameter("frame_id").get_parameter_value().string_value
        )
        self.model_path = (
            self.get_parameter("model_path").get_parameter_value().string_value
        )

```

```

self.debug_theta = (
    self.get_parameter("debug_theta").get_parameter_value().bool_value
)

if self.prediction_horizon <= 0.0:
    self.prediction_horizon = 3.0
if self.prediction_dt <= 0.0:
    self.prediction_dt = 0.1

self.num_steps = max(1, int(self.prediction_horizon / self.prediction_dt))

# ----- 加载 LSTM 模型 -----
self.device = torch.device("cpu") # 为稳定起见，默认用 CPU
self.model, self.obs_len, self.dt_model =
self._load_lstm_model(self.model_path)
self.min_history_len = self.obs_len # 至少需要 obs_len 帧历史

if abs(self.dt_model - self.prediction_dt) > 1e-3:
    self.get_logger().warn(
        f'模型训练 dt={self.dt_model:.3f}s 与当前
prediction_dt={self.prediction_dt:.3f}s 不一致，"
        f'预测时间轴会有点缩放，一般问题不大。"
    )

# 每个障碍物的历史观测：id -> List[(t, x, y)]
self.histories: Dict[str, List[Tuple[float, float, float]]] = {}

# ----- 订阅 & 发布 -----
self.sub_poses = self.create_subscription(
    ObjectPoseArray,
    "/object_detection/object_poses",
    self.object_pose_callback,
    50,
)

self.pub_traj = self.create_publisher(
    ObstacleTrajectoryArray,
    "/obstacle_trajectory",
    10,
)

```

```

# 0 调试话题: Float32MultiArray
# data = [axis_flag, min, max, fixed, speed, s_amp, s_waves]
self.theta_pub = self.create_publisher(
    Float32MultiArray,
    "/obstacle_theta_debug",
    10,
)

# 定时预测发布: 步长用 prediction_dt
self.timer = self.create_timer(self.prediction_dt, self.timer_callback)

self.get_logger().info(
    "🚀 LSTM 解析参数式障碍物轨迹预测节点已启动\n"
    f" - prediction_horizon = {self.prediction_horizon:.2f} s\n"
    f" - prediction_dt      = {self.prediction_dt:.3f} s\n"
    f" - num_steps           = {self.num_steps}\n"
    f" - frame_id            = {self.frame_id}\n"
    f" - model_path           = {self.model_path}\n"
    f" - LSTM obs_len        = {self.obs_len}\n"
    f" - LSTM dt_model       = {self.dt_model:.3f} s\n"
    f" - debug_theta         = {self.debug_theta}"
)

# =====
# 模型加载
# =====
def _load_lstm_model(self, path: str):
    try:
        ckpt = torch.load(path, map_location=self.device)
    except Exception as e:
        self.get_logger().error(f"❌ 加载 LSTM 权重失败: {path}\n{e}")
        raise

    obs_len = ckpt.get("obs_len", 16)
    dt_model = ckpt.get("dt", 0.1)
    hidden_dim = ckpt.get("hidden_dim", 64)
    num_layers = ckpt.get("num_layers", 2)

```

```

param_dim = ckpt.get("param_dim", 7)

model = LSTMThetaModel(
    input_dim=2,
    hidden_dim=hidden_dim,
    num_layers=num_layers,
    param_dim=param_dim,
).to(self.device)

model.load_state_dict(ckpt["model_state_dict"])
model.eval()

self.get_logger().info(
    f'✅ 成功加载 LSTM 权重: obs_len={obs_len}, dt={dt_model:.3f}, "
    f"hidden_dim={hidden_dim}, num_layers={num_layers},
param_dim={param_dim}"
)

return model, obs_len, dt_model

# =====
# 工具函数：1D 线段往返反射运动
# =====
@staticmethod
def _reflect_1d(x0: float, v: float, min_val: float, max_val: float, t: float) -> float:
    """
    在 [min_val, max_val] 上做 1D 往返运动的解析式解：
    - 初始位置 x0
    - 匀速 v (m/s)，可以为正或负
    - 时间 t >= 0
    返回：x(t)，始终处于 [min_val, max_val] 内
    """
    L = max_val - min_val
    if L <= 1e-6:
        return x0

    x0_clamped = min(max(x0, min_val), max_val)
    rel0 = x0_clamped - min_val

```

```

s = rel0 + v * t
cycle = 2.0 * L

m = math.fmod(s, cycle)
if m < 0.0:
    m += cycle

if m <= L:
    rel = m
else:
    rel = 2.0 * L - m

return min_val + rel

# =====
# 回调：接收 Gazebo 发布的障碍物位姿
# =====
def object_pose_callback(self, msg: ObjectPoseArray):
    # 统一用 header 时间戳（秒）
    t_msg = msg.header.stamp.sec + msg.header.stamp.nanosec * 1e-9

    for obj in msg.objects:
        if obj.type != "obstacle_dynamic":
            continue

        obs_id = obj.id
        x = float(obj.pose.position.x)
        y = float(obj.pose.position.y)

        history = self.histories.get(obs_id)
        if history is None:
            history = []
            self.histories[obs_id] = history

        history.append((t_msg, x, y))

        # 只保留最近 obs_len 帧
        if len(history) > self.obs_len:
            del history[0 : len(history) - self.obs_len]
```

```

# =====
# 核心：利用 LSTM 输出解析  $\theta$ ，再生成未来轨迹
# =====
def _predict_theta_from_history(self, obs_id: str, history: List[Tuple[float, float,
float]]):
    """
    输入：某个障碍物的历史观测列表 [(t,x,y), ...]，长度 >= obs_len
    输出：解析参数  $\theta$  = (axis, min_val, max_val, fixed, speed, s_amp, s_waves)
    做了基本的 clamp，保证参数稳定。
    """
    assert len(history) >= self.obs_len

    # 取最近 obs_len 帧，按时间顺序构造 (x,y) 序列
    recent = history[-self.obs_len:]
    coords = [[x, y] for (_, x, y) in recent]

    obs_tensor = torch.tensor(
        [coords], dtype=torch.float32, device=self.device
    ) # (1, obs_len, 2)

    with torch.no_grad():
        theta_pred = self.model(obs_tensor)[0] # (7,)
        theta_pred = theta_pred.cpu().tolist()

    axis_flag = theta_pred[0]
    raw_min = theta_pred[1]
    raw_max = theta_pred[2]
    fixed = theta_pred[3]
    speed = abs(theta_pred[4]) # 速度取绝对值
    s_amp = max(0.0, theta_pred[5])
    s_waves = max(0.0, theta_pred[6])

    # 轴向：<0.5 → x 轴；>=0.5 → y 轴
    axis = "x" if axis_flag < 0.5 else "y"

    # min / max 排序，确保 min <= max
    min_val = min(raw_min, raw_max)
    max_val = max(raw_min, raw_max)

```

```
# 基础 clamp: 防止偶发异常
min_val = max(-20.0, min_val)
max_val = min(20.0, max_val)
if min_val > max_val:
    min_val, max_val = max_val, min_val

fixed = max(-10.0, min(10.0, fixed))

# 速度 clamp: 0.1 ~ 0.8 m/s
speed = max(0.1, min(speed, 0.8))

# S 型参数 clamp
s_amp = max(0.0, min(s_amp, 2.0))
s_waves = max(0.0, min(s_waves, 3.0))

# 📌 日志监控 0
if self.debug_theta:
    self.get_logger().info(
        f'[theta] id={obs_id}  axis={axis} '
        f'min={min_val:.3f}, max={max_val:.3f}, fixed={fixed:.3f}, '
        f'speed={speed:.3f}, s_amp={s_amp:.3f}, s_waves={s_waves:.3f}'
    )

# 📌 发布到调试话题 /obstacle_theta_debug
msg = Float32MultiArray()
# data 顺序固定为 [axis_flag, min, max, fixed, speed, s_amp, s_waves]
axis_flag_out = 0.0 if axis == "x" else 1.0
msg.data = [
    float(axis_flag_out),
    float(min_val),
    float(max_val),
    float(fixed),
    float(speed),
    float(s_amp),
    float(s_waves),
]
self.theta_pub.publish(msg)
```

```

    return axis, min_val, max_val, fixed, speed, s_amp, s_waves

def _generate_future_trajectory(
    self,
    obs_id: str,
    history: List[Tuple[float, float, float]],
) -> ObstacleTrajectory:
    """
    核心函数：
    - 用历史轨迹 -> LSTM 拟合  $\theta$ 
    - 用  $\theta$  + 当前最新位置 -> 生成未来轨迹
    """
    axis, min_val, max_val, fixed, speed, s_amp, s_waves = \
        self._predict_theta_from_history(obs_id, history)

    # 当前最新观测
    _, cur_x, cur_y = history[-1]

    # 根据主运动方向和最近两帧观测估计运动方向符号
    if len(history) >= 2:
        _, prev_x, prev_y = history[-2]
    else:
        prev_x, prev_y = cur_x, cur_y

    if axis == "x":
        d = cur_x - prev_x
        axis_pos0 = cur_x
    else:
        d = cur_y - prev_y
        axis_pos0 = cur_y

    if abs(d) < 1e-4:
        # 如果最近两帧几乎不动，就用“离哪头近就朝哪头”的规则估计方向
        if abs(max_val - axis_pos0) < abs(axis_pos0 - min_val):
            dir_sign = -1.0
        else:
            dir_sign = 1.0
    else:
        dir_sign = 1.0 if d >= 0.0 else -1.0

```

```

v = dir_sign * speed # 带符号速度

traj_msg = ObstacleTrajectory()
traj_msg.id = obs_id
traj_msg.type = "obstacle_dynamic"

L = max_val - min_val if max_val > min_val else 0.0

# 从 t = prediction_dt 开始往前推
for k in range(1, self.num_steps + 1):
    t = self.prediction_dt * float(k)

    # 1D 反射运动
    coord = self._reflect_1d(axis_pos0, v, min_val, max_val, t)

    pose = Pose()
    if axis == "x":
        x = coord
        y_center = fixed
    else:
        x = fixed
        y_center = coord

    # S 型偏移（只有 s_amp > 0 时生效，一般对应 obstacle4）
    if s_amp > 1e-3 and L > 1e-6:
        phase = (coord - min_val) / L
        phase = max(0.0, min(1.0, phase))
        angle = math.pi * (2.0 * phase - 1.0) * s_waves
        offset = s_amp * math.sin(angle)
    else:
        offset = 0.0

    if axis == "x":
        y = y_center + offset
    else:
        # 如果以后需要“沿 y 轴 + S 型”，可以在 x 上加 offset
        y = y_center

    pose.position.x = x
    pose.position.y = y

```

```

        pose.position.z = 0.3
        pose.orientation.x = 0.0
        pose.orientation.y = 0.0
        pose.orientation.z = 0.0
        pose.orientation.w = 1.0

        traj_msg.future_poses.append(pose)
        traj_msg.future_times.append(float(t))

    return traj_msg

# =====
# 定时器：遍历所有障碍物，根据历史生成预测轨迹并发布
# =====
def timer_callback(self):
    if not self.histories:
        return

    traj_array_msg = ObstacleTrajectoryArray()
    traj_array_msg.header = Header()
    traj_array_msg.header.stamp = self.get_clock().now().to_msg()
    traj_array_msg.header.frame_id = self.frame_id

    for obs_id, history in self.histories.items():
        # 历史长度不够，先不预测
        if len(history) < self.min_history_len:
            continue

        try:
            traj_msg = self._generate_future_trajectory(obs_id, history)
            traj_array_msg.trajectories.append(traj_msg)
        except Exception as e:
            self.get_logger().warn(
                f"⚠️ 生成障碍物 {obs_id} 未来轨迹时出错: {e}"
            )

    if traj_array_msg.trajectories:
        self.pub_traj.publish(traj_array_msg)
        ids = [t.id for t in traj_array_msg.trajectories]
```

```

        self.get_logger().debug(
            f'⚡ 已发布 LSTM 解析未来轨迹: {ids}'
        )

def main(args=None):
    rclpy.init(args=args)
    node = LSTMPredictorNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info(" LSTM 解析预测节点手动退出")
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

4.4 YOLOv8 视觉识别模块关键代码 (yolov8_vision)

```

# 文件: yolov8_vision/yolov8_vision/yolo_node.py

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
from ultralytics import YOLO
import cv2

class YoloV8Node(Node):
    def __init__(self):
        super().__init__("yolov8_node")

        # 模型加载 (best.pt 为训练得到的权重)
        model_path = self.declare_parameter(
            "model_path",
            "/home/ros/dev_ws/src/yolov8_vision/models/best.pt"

```

```
    ).value
    self.model = YOLO(model_path)

    # ROS 接口
    self.bridge = CvBridge()
    self.image_sub = self.create_subscription(
        Image, "/camera/image_raw", self.image_callback, 10
    )
    self.image_pub = self.create_publisher(
        Image, "/yolov8/annotated_image", 10
    )

    self.get_logger().info(f"YOLOv8 模型已加载: {model_path}")

def image_callback(self, msg: Image):
    # 转换图像格式
    frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")

    # YOLOv8 推理
    results = self.model.predict(frame, imgsz=640, conf=0.5, verbose=False)

    # 绘制检测框
    annotated = results[0].plot()

    # 发布标注图像
    out_msg = self.bridge.cv2_to_imgmsg(annotated, "bgr8")
    self.image_pub.publish(out_msg)

def main(args=None):
    rclpy.init(args=args)
    node = YoloV8Node()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

参考文献

- [1] Llama.cpp 轻量级大语言模型部署指南. <https://github.com/ggml-org/llama.cpp>
- [2] Python threading 模块线程安全编程规范.
<https://docs.python.org/3/library/threading.html>
- [3] Foxglove Studio 可视化工具用户手册. <https://docs.foxglove.dev/>
- [4] Requests 库 HTTP 请求异常处理最佳实践. <https://requests.readthedocs.io/>
- [5] Williams, G., et al. Model Predictive Path Integral Control for Fast, Aggressive Driving. ICRA, 2016.
- [6] Rhinehart, N., et al. Deep Imitative Models for Flexible Inference, Planning, and Control. ICLR, 2020.
- [7] Pfeiffer, M., et al. Predicting Trajectory Distributions for Autonomous Robots. ICRA, 2017.
- [8] Everett, M., et al. Motion Planning Among Dynamic, Decision-Making Agents. RSS, 2020.
- [9] ROS2 Navigation Stack Documentation (Nav2).
- [10] Xu et al. Safe Reinforcement Learning via Control Barrier Functions. NeurIPS, 2021.
- [11] Levine, S. End-to-End Training of Deep Visuomotor Policies. JMLR, 2016.
- [12] Ultralytics. YOLOv8 目标检测模型官方文档.
<https://docs.ultralytics.com/models/yolov8/>
- [13] Ultralytics. YOLOv8 开源实现与使用示例 (GitHub 仓库).
<https://github.com/ultralytics/ultralytics>
- [14] Roboflow. YOLOv8 Object Detection Model: Overview & Usage.
<https://roboflow.com/model/yolov8/>